

南京工业大学

2026 届毕业设计（论文）

题 目：面向 F2FS 文件系统的
Large Folios 机制研究与实现

专 业：人工智能

班 级：智能 2202

姓 名：赵南哲

指导老师：万夕里

起讫日期：2026 年 3 月 5 日

至 2026 年 6 月 10 日

2026 年 5 月

面向 F2FS 文件系统的 Large Folios 机制研究与实现

摘 要

随着服务器内存容量和 NVMe 存储设备的发展，Linux 内核长期使用 4 KiB page 作为基础管理单元的设计，在页缓存和文件系统 I/O 路径中逐渐暴露出可扩展性问题。为适应大内存和大块顺序 I/O，内核社区引入动态大页:Large Folio，并推动 iomap 取代传统 buffer_head 模型。该框架以区间方式描述文件映射和写回状态，为页缓存提供更大的处理粒度。本文围绕 Large Folio 与 iomap 在安卓常用闪存文件系统 F2FS 页缓存路径中的适配展开。论文结合 F2FS 的日志结构、压缩簇和原子写特性，分析大页缓存粒度与文件系统块状态之间的错配问题，并给出工程实现方案。

在实现上，本文基于 iomap buffered write 和写回通路，补充 F2FS 对 Large Folio 的声明、分配、逐块脏页追踪和写回边界控制。该实现使一个 Large Folio 内只有实际修改的文件系统块进入 I/O 提交路径。实验结果表明，在顺序读写负载下，该方案减少了页缓存元数据和锁竞争开销，降低了映射与写回路径中的 CPU 消耗，并提升了 F2FS 大块 I/O 的吞吐表现。

关键词：动态大页；iomap；文件系统；页缓存；写回

Research and Implementation of the Large Folios Mechanism for the F2FS File System

Abstract

Large-memory servers and NVMe storage expose scalability limits in the Linux page cache and filesystem I/O stack. Traditional 4KB page granularity becomes inefficient under large working sets and sequential I/O. Linux has introduced dynamic grained huge page: Large Folios, and iomap to improve page-cache management. This thesis studies how these mechanisms can be integrated into F2FS. The target filesystem includes log-structured allocation, compression clusters, and atomic write semantics. The main issue is the mismatch between large page-cache units and per-block filesystem state. The implementation extends the F2FS buffered write and writeback paths. It adds per-block dirty tracking and controls writeback boundaries inside a Large Folio. Error rollback, compressed writeback, and atomic file paths are also adjusted. These changes match iomap delalloc release and state-machine semantics. Experiments show lower metadata overhead, reduced lock contention, and improved large-block I/O throughput on F2FS.

Key Words: dynamic grained huge page; iomap; filesystem; page cache; writeback

目 录

摘要	2
ABSTRACT	3
第一章 研究背景	1
1.1 内存管理机制的演进	1
1.1.1 页表与 TLB 的工作原理	1
1.1.2 4KB 页面在大内存系统中的扩展性问题	1
1.2 Linux 文件系统 I/O 栈概述	2
1.2.1 页缓存与 VFS 层交互	2
1.2.2 传统 buffer_head 映射机制及其局限性	4
1.3 F2FS 文件系统架构	6
1.3.1 日志结构文件系统 (LFS) 与闪存感知设计	6
1.3.2 数据布局、Segment 结构与 Checkpoint 机制	7
第二章 引入 Folio 和 iomap 的意义	8
2.1 Struct Folio: 新一代内存管理单元	8
2.1.1 Linux 传统大页机制回顾	8
2.1.2 Folio 在内核中的演进现状与趋势	10
2.2 iomap 框架: 现代化的 I/O 路径	11
2.2.1 iomap 数据结构与接口定义	12
2.2.2 迭代器模式与区间映射优势	14
第三章 F2FS 支持 Large Folios 的核心难点	15
3.1 块映射与 I/O 提交的粒度失配	16
3.1.1 逐页处理带来的 CPU 开销	16
3.1.2 Large Folio 子区间多 BIO 提交与 I/O 生命周期失配	16
3.1.3 空洞文件与 I/O 区间划分的一致性	19
3.2 关键数据结构的冲突与兼容	20
3.2.1 私有字段复用带来的状态冲突	20
3.2.2 状态误判导致的崩溃风险	20
3.2.3 I/O 描述粒度限制	23
3.3 压缩文件场景下的复杂性	23
3.3.1 压缩簇与 Folio 边界的不对齐	23

3.3.2 混合 I/O 模型（压缩与非压缩混排）	24
3.4 脏页写回中的写放大风险	24
3.4.1 F2FS 原有写回模型与写放大根源	24
3.4.2 Large Folio 与压缩写回的架构性挑战	25
第四章 F2FS Large Folio 缓冲 I/O 设计与实现	26
4.1 设计目标与总体思路	26
4.2 基于区间映射的普通文件 I/O 路径	26
4.2.1 F2FS 块映射与 iomap 字节区间的粒度差异	27
4.2.2 块区间到 iomap 区间的转换	27
4.2.3 普通文件读取路径	29
4.2.4 普通文件写入路径与空间预分配	30
4.3 面向 Large Folio 的状态管理	32
4.3.1 统一状态结构	32
4.3.2 状态分配与旧私有状态迁移	34
4.3.3 读完成计数	37
4.4 脏页写回路径中的粒度控制	38
4.4.1 从逐页写回到 Folio 内脏区定位	38
4.4.2 子范围提交与重试边界	41
4.4.3 压缩上下文中的延迟解锁	45
4.5 GC 路径中的 Large Folio 适配	46
4.5.1 GC 搬运中的私有状态	46
4.5.2 GC 读路径与 bio 提交偏移	47
4.5.3 前台搬运中的写回控制	48
4.6 压缩文件场景下的扩展处理	49
4.6.1 压缩读取中的簇级映射与完成计数	49
4.6.2 压缩写入中的 Large Folio 准备	56
4.6.3 压缩写回中的簇级提交	57
第五章 项目性能测试	58
5.1 测试环境与方法论	58
5.1.1 顺序读写测试的批量调度	58
5.1.2 单轮 FIO 执行与缓存控制	63
5.2 性能测试结果与分析	64
5.2.1 QEMU 虚拟机性能测试	64

5.2.2 树莓派 5 性能测试.....	68
5.3 数据处理与可视化脚本.....	72
5.3.1 FIO 日志的结构化解析	72
5.3.2 测试结果的图表生成	75
结语.....	77
参考文献	78
致谢.....	80

第一章 研究背景

1.1 内存管理机制的演进

本文关注文件系统页缓存路径中的粒度问题。Linux 长期以 4 KiB page 作为页缓存和内存管理的基本单位，当一次文件访问覆盖较大的连续区间时，页缓存仍需要围绕多个小页组织状态、查找映射并提交 I/O。对 F2FS 而言，这种粒度差异会继续传递到块映射、写回和压缩文件路径中，因此本文将 Large Folios 作为后续改造的入口。

1.1.1 页表与 TLB 的工作原理

页表负责维护虚拟地址到物理地址的映射，转换后备缓冲区（Translation Lookaside Buffer, TLB）用于缓存近期使用的地址转换结果。本文不展开处理器体系结构细节，只保留一个与后文相关的事实：页越小，覆盖同一内存范围所需的页表项和 TLB 项通常越多。

在 4 KiB page 配置下，较大的工作集会被拆成大量页表项。TLB 容量有限时，地址转换结果更容易被替换，后续访问需要重新进行 page walk。Large Folios 并不直接改变文件系统语义，但它可以让页缓存一次覆盖更大的连续文件范围，减少后续 I/O 路径围绕小页反复处理的次数。

1.1.2 4KB 页面在大内存系统中的扩展性问题

Weisberg 和 Wiseman 将 4 KiB page 的优点和代价放在同一框架下讨论：小页有利于控制外部碎片，但在大内存系统中会缩小 TLB 覆盖范围，并增加页表遍历开销[1]。后续关于虚拟化和云环境的研究也把这一问题放到更大的工作集和更多地址空间切换场景中讨论[2][4]。本文只取其中与页缓存有关的一点：当上层 I/O 已经形成较大连续访问时，4 KiB 粒度会把映射、状态和提交过程拆成更多小步骤。

Patil 对多核体系结构上的 TLB 与 page walk 开销进行了评估，相关结论说明 superpage 可以通过扩大 TLB 覆盖范围减少 miss 和 page walk 次数[5]。Sites 也在技术文章中讨论过 x86-64 多级页表下 TLB miss 的额外访存开销[6]。这些研究解释了大页机制的体系结构动机，但本文后续重点放在文件系统页缓存如何承接更大的 folio 粒度。

4 KiB 粒度还会增加内核元数据数量。Martins 在 Oracle Linux 内核博客中估算，x86 系统上每个 4 KiB 页对应一个约 64 字节的 struct page，1 TB 内存约需要 16 GB

的页元数据[7]。Corbet 在分析 struct page 重构时也指出，内核需要为每个物理页维护 page 结构[8]。页数量增加后，回收、迁移和写回等路径都要处理更多元数据。

Wilcox 在 2023 年的 “The state of the page in 2023” 报告中强调，长期围绕 4 KiB page 维护系统会放大内核遍历 page 元数据的成本[11]。对文件系统而言，这一问题会在页缓存读写、回收和写回路径中表现出来：如果一次连续 I/O 被拆成许多基本页，文件系统侧也需要重复维护锁、状态位和映射结果。

页回收路径也体现了类似问题。Wilcox 在相关补丁说明中提到，page reclaim 的 CPU 开销和回收对象选择仍有优化空间[10]；Corbet 在分析 MGLRU 时指出，传统 reclaim 需要扫描页并结合反向映射检查页表项[9]。这些工作说明，减少基本页数量本身就能降低部分内核路径的管理压力。

1.2 Linux 文件系统 I/O 栈概述

本文只讨论 buffered I/O 中与页缓存相关的路径。一次普通文件读取通常从 VFS 进入具体文件系统，再经过页缓存查找、预读和块层提交。后续改造集中在页缓存与 F2FS 之间的接口，而不是整个 Linux I/O 栈的完整调度过程。

1.2.1 页缓存与 VFS 层交互

VFS 通过 struct file_operations 将系统调用转入具体文件系统。以读取为例，F2FS 的 read_iter 最终会回到通用的 filemap_read 路径；如果目标数据不在页缓存中，内核再通过 readahead 触发文件系统读取。本文关注的 Large Folio 改造，正发生在 filemap_read、readahead 与 F2FS address_space 操作之间。

以下代码给出从 VFS 层到具体文件系统（以 F2FS 为例）的读取调用路径。可以看到，内核通过 generic_file_read_iter 最终调用 filemap_read，并触发 readahead 预读机制：

```
// File: fs/read_write.c & fs/f2fs/file.c & mm/filemap.c

// vfs read entry
ssize_t vfs_read(struct file *file, char __user *buf, size_t count, loff_t *pos)
{
    if (file->f_op->read)
        ret = file->f_op->read(file, buf, count, pos);
    else if (file->f_op->read_iter)
```



```

        ret = new_sync_read(file, buf, count, pos);
    else
        ret = -EINVAL;

    /*...*/
    return ret;
}

// F2FS's specific implementation of file_operations
static const struct file_operations f2fs_file_operations = {
    .read_iter = f2fs_file_read_iter,
    .write_iter = f2fs_file_write_iter,
    // ...
};

// read_iter
static ssize_t f2fs_file_read_iter(struct kiocb *iocb, struct iov_iter *to)
{
    // ... direct IO
    return generic_file_read_iter(iocb, to);
}

//filemap.c
ssize_t generic_file_read_iter(struct kiocb *iocb, struct iov_iter *iter)
{
    // ...
    return filemap_read(iocb, iter, 0);
}

// in filemap_read->filemap_get_pages->page_cache_ra_order
static void read_pages(struct readahead_control *rac)
{
    //...
    if (aops->readahead) {
        aops->readahead(rac);
        /* Clean up the remaining folios. */
        while ((folio = readahead_folio(rac)) != NULL) {
            folio_get(folio);
            filemap_remove_folio(folio);
            folio_unlock(folio);
            folio_put(folio);
        }
    } else {
        while ((folio = readahead_folio(rac)) != NULL)
            aops->read_folio(rac->file, folio);
    }

    //...
}

// F2FS's readahead function

```

```
const struct address_space_operations f2fs_dblock_aops = {
    .readahead = f2fs_readahead,
};
```

因此，页缓存并不是单纯的缓存层。它决定一次缺页读取分配多大的 folio、如何把读取范围交给文件系统，以及 I/O 完成后何时把 folio 标记为 uptodate。F2FS 支持 Large Folios 时，必须在这些边界上提供与大 folio 匹配的映射和完成语义。

1.2.2 传统 buffer_head 映射机制及其局限性

在 iomap 普及之前，许多 Linux 文件系统使用 buffer_head 记录文件系统块与页缓存之间的关系。buffer_head 保存块号、块大小、块设备、关联的 struct page 和 I/O 完成回调等字段[14]。这种模型的基本特点是：页缓存中的一个页可以挂接多个块状态对象，文件系统再围绕这些对象维护 uptodate、dirty 和提交状态。

buffer_head 的问题不在于它不能描述块映射，而在于它把块状态挂回到 page 之上。当页内包含多个文件系统块时，读写路径需要维护一组 buffer_head 状态；当页缓存粒度扩大到 Large Folio 后，这种组织方式又会把状态管理拆回小块链表。iomap 的意义正在于把映射结果表达为文件字节区间，减少新路径对每块一个状态对象的依赖[16]。

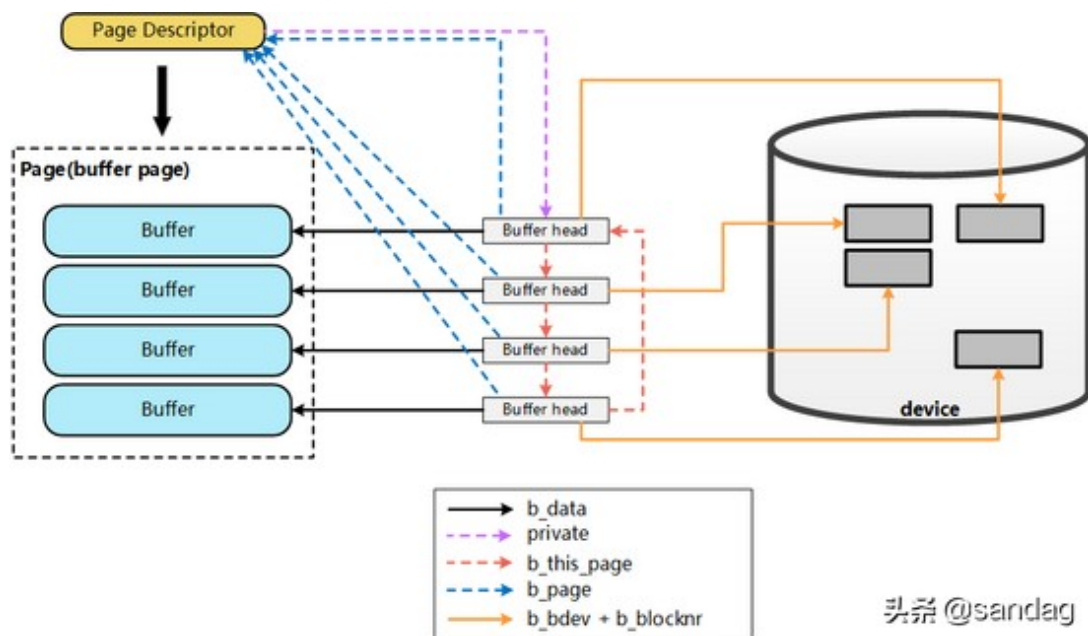


图 3-1 buffer_head 结构体示意图

块层 BIO 已经可以承载不同大小的 I/O，请求合并和调度也主要在块层完成。此时 buffer_head 更像是页内块状态的遗留容器，而不是唯一的 I/O 提交单位。内核文

档也指出，旧 buffered I/O 模型依赖 `buffer_head` 链表维护每个块的 `uptodate`、`dirty` 状态和写回状态，效率较低[16]。

对本文来说，`buffer_head` 的主要影响有两点。第一，页内块状态需要额外对象保存；块越小，状态对象越多[14][16]。第二，I/O、刷新和回收路径需要遍历这些对象，并逐个更新标志位或组织 BIO[14]。在 Large Folio 场景下，如果仍沿用这种组织方式，一个 folio 内部的状态管理会继续被拆成多个小块处理。

可扩展性问题还体现在粒度关系变化之后。相关内核文档指出，`block size` 大于 `page size` 的场景需要摆脱 `buffer_head` 路径，转向 `iomap` 的区间化映射接口[16]。Large Folio 使页缓存对象本身变大，虽然方向相反，但同样要求文件系统用更清晰的区间语义描述映射和状态。`iomap` 正是这一方向上的通用接口。

因此，本节不把 `buffer_head` 视为单个函数替换问题，而把它放在粒度变化中理解：页缓存希望一次管理更大的内存范围，块层也可以提交更大的连续 I/O，文件系统侧则需要提供能覆盖连续区间的映射结果。已有关于大页、内存连续性和高性能 SSD 的研究也说明，增大基本处理粒度可以减少元数据和地址转换成本[18]。

1.3 F2FS 文件系统架构

F2FS（Flash-Friendly File System）是面向 NAND 闪存设计的日志结构文件系统，最初由 FAST'15 论文系统提出[19]。它运行在已经由 Flash Translation Layer（FTL）管理的块设备之上[20]。因此，F2FS 的核心设计不是直接控制裸闪存，而是在块设备抽象下尽量保持顺序写入，减少随机覆盖更新带来的写放大[19][20]。

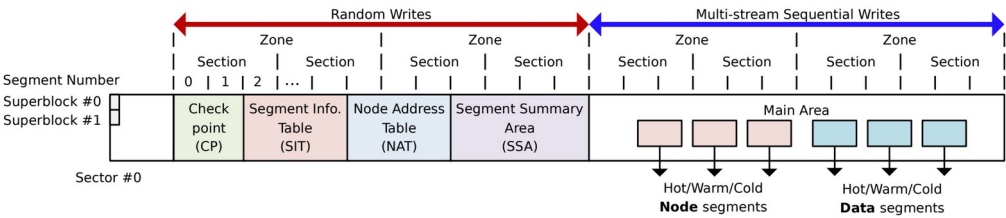


图 1-1 F2FS 磁盘布局架构图（Superblock / Checkpoint / SIT / NAT / SSA / Main Area）

1.3.1 日志结构文件系统 (LFS) 与闪存感知设计

NAND 闪存不能像磁盘一样在原位置任意覆盖。通常情况下，数据按页写入、按擦除块回收；当上层只改动少量页时，FTL 可能需要把有效数据读出、合并，再写入

新的物理位置[19][21]。这种读改写过程会放大实际写入量，也就是写放大（Write Amplification）[19]。

闪存还需要磨损均衡（Wear-Leveling）来分散擦写压力。若文件系统长期产生小随机写，FTL 在顺序调度、垃圾回收（Garbage Collection, GC）和磨损均衡之间需要做更多折中[19]。相关研究和工程经验表明，持续随机写会影响 SSD 后续吞吐表现[13][19]。

F2FS 采用日志结构思想：用户数据和大部分元数据通过顺序追加写入，旧块随后被标记为无效[19]。这种方式把文件系统层面的覆盖更新转换为追加写，便于 FTL 将连续写请求映射到较连续的物理空间，也有利于后续回收无效块[19][21]。

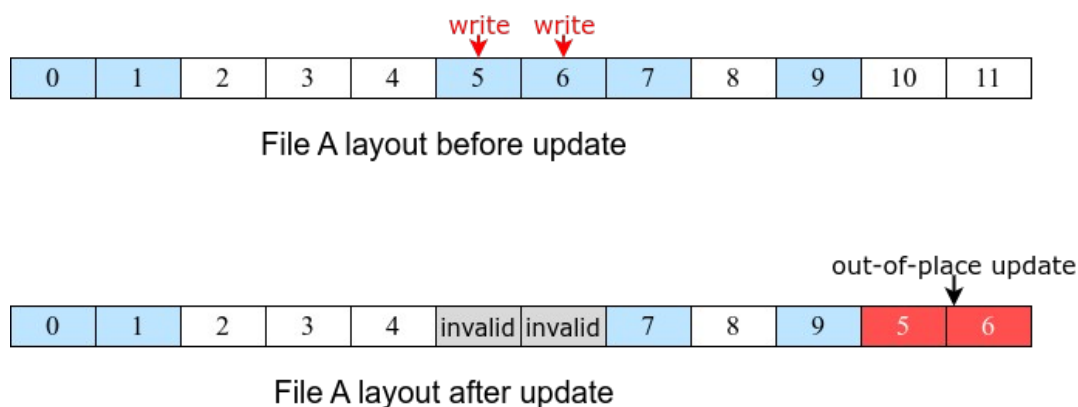


图 1-2 F2FS 日志结构写入示意图

F2FS 还在日志结构之上加入冷热数据分离和多日志支持。不同更新频率、不同类型的数据会被写入不同日志流或物理区域[18][19]。这一设计使频繁更新的数据更集中，较少更新的数据则尽量保持稳定，从而降低后续 GC 搬运有效数据的比例。

顺序追加写也解释了本文为什么关注页缓存粒度。如果页缓存能够向 F2FS 提供更大的连续写入范围，文件系统更容易形成连续的日志写入；若页缓存仍按 4 KiB 小页反复提交，F2FS 的日志结构优势会被上层粒度拆分削弱。已有关于大粒度 I/O 的研究也指出，较大的处理单位有助于减少元数据和地址转换开销[18][19]。

1.3.2 数据布局、Segment 结构与 Checkpoint 机制

F2FS 的布局介绍只保留与后文有关的部分：用户数据主要写入 Main Area，段状态由 SIT 记录，节点地址由 NAT 维护，SSA 则保存段摘要信息[19][20]。这些结构决定了 F2FS 在搬移有效块、更新节点地址和恢复日志时，不能只考虑页缓存状态，还必须维护文件系统自己的块级元数据。

F2FS 的空间管理以 block、segment、section 和 zone 分层组织[19][20]。在常见配置中，segment 由连续块组成，GC 也倾向于围绕 section 或 zone 等更大的范围选择和搬移有效数据[19][21]。这种布局与本文问题直接相关：当页缓存一次覆盖更大的连续区间时，F2FS 更容易把上层请求转化为连续映射和连续写入。

SIT、NAT 和 SSA 分别服务于段利用率、节点地址和段摘要管理[19][20]。GC 搬移数据时，需要根据 SIT 选择候选段，通过 NAT 更新节点位置，并借助 SSA 识别段内有效数据。后文讨论 GC 与 Large Folio 适配时，关键就在于页缓存中的 folio 状态不能破坏这些块级元数据更新。

Checkpoint 机制用于记录一组一致的文件系统元数据，包括 SIT/NAT 的增量变化、当前日志头位置和脏段信息[19]。挂载时，F2FS 选择有效的 Checkpoint，并结合 SSA 中的段摘要进行 Roll-forward，以恢复尚未完全落盘的日志片段[19][20]。本文后续修改读写路径时，需要保证 Large Folio 的状态变化不会绕过这些一致性边界。

F2FS 的文件系统块仍以 4 KiB 为基本粒度，而内核和移动平台已经开始支持 16 KiB 甚至更大的 page size 配置[23]。因此，本文面对的不是单纯的内存大页问题，而是页缓存 folio、F2FS block、压缩簇和 GC 搬移范围之间的粒度协调问题。后续章节围绕这一协调关系展开。

第二章 引入 Folio 和 iomap 的意义

随着存储硬件性能的指数级增长，Linux 内核传统的内存管理和文件系统 I/O 路径日益显现出疲态。为了应对“大内存”和“高性能存储”的双重挑战，内核社区引入了 Struct Folio 和 iomap 框架。本章将详细阐述这两项技术的引入背景、设计哲学及其带来的性能变革。

2.1 Struct Folio：新一代内存管理单元

2.1.1 Linux 传统大页机制回顾

在 Folio 出现之前，Linux 内核针对“比 4KB 更大”的内存单元，实际上走了三条路线：面向用户空间显式使用的 hugetlbfs。尝试自动折叠匿名页/页缓存的透明大页（Transparent Huge Pages, THP），以及完全在内核内部使用的复合页（Compound Page）。

hugetlbfs 通过专门的文件系统和一套显式分配接口向应用暴露预留的大页池。典型用法包括挂载 hugetlbfs、在该挂载点上 mmap 带有 MAP_HUGETLB 标志的文件，

或通过 `shmget(SHM_HUGETLB)` 申请共享内存。运维侧需要事先通过 `/proc/sys/vm/nr_hugepages` 等接口预留物理连续的大页，应用侧则必须针对性地修改代码，才能享受到 TLB 命中率提升带来的收益。

然而，随着内核代码规模和硬件场景扩大，`hugetlbfs` 的维护成本和语义复杂度开始受到内核社区质疑。Matthew Wilcox 在 2023 年的讨论中表示，`hugetlbfs` 需要被逐步替代，但这是一项规模很大的工作[11][12]。他指出，`hugetlbfs` 历史上叠加了 `reservation`、页表共享等专有特性。这些语义与通用内存管理代码耦合较深，也难以复用到其他子系统。

相较之下，透明大页（THP）试图在不修改应用的前提下自动利用大页。内核通过后台折叠线程和页错误处理逻辑，将相邻的 4KB 匿名页或页缓存页折叠为 2MB 的 PMD 级映射。THP 同时提供 `madvise(MADV_HUGEPAGE / MADV_NOHUGEPAGE)` 等 hint 接口，允许应用按 VMA 粒度表达大页偏好。

从实践反馈来看，固定 2MB 的 THP 同样暴露出一系列问题。首先，它在许多负载下会产生明显的内部碎片：应用的实际访问模式往往并未充分利用整整 2MB，但只要少量 4KB 子页被写入，整个大页就会常驻内存。其次，为了维持这些大页，内核需要频繁执行内存压缩（compaction）来寻找连续的 2MB 物理空间。在长期运行的系统上，这种高阶物理块分配本身就越来越困难。一旦物理内存被碎片化，THP 分配失败率会显著升高，系统在大页折叠与回退到 4KB 小页之间反复抖动。引入额外的 CPU 开销和延迟抖动[13]。

在围绕 THP 内部碎片的讨论中，有内核开发者指出，问题的本质在于 `internal fragmentation`。也就是说，THP 内部可能存在从未被写入的 4KB 子页[10][13]。在现实负载中，应用实际访问的工作集可能小于 2MB，但 THP 仍需要为整块 2MB 大页分配物理内存并维护页表映射。

与 `hugetlbfs` 和 THP 面向用户空间的大页不同，复合页（Compound Page）是内核内部用来“拼接”多个连续 4KB 页的通用机制。它通过让一个 `head page` 持有整体元数据，并将其后的若干 `tail page` 标记为附属，向上暴露出一个比 4KB 更大的逻辑页。早期的 THP 实现、文件系统页缓存的大页化、高阶伙伴分配等，都不同程度地依赖于这一机制。从抽象上看，复合页为“动态大小的大页”提供了基础设施。

复合页的设计在接口层面也存在隐患。许多内存管理 API 仍以 `struct page *` 作为参数，调用者拿到的指针可能指向 `head page`，也可能指向 `tail page`。如果代码在 `tail`

page 上错误地加锁、调整引用计数或挂接 LRU，就可能破坏复合页内部结构。因此，大量内核代码需要反复调用 `compound_head()`、`PageHead()`、`PageTail()` 等辅助函数来判断 page 角色，这增加了代码审查和维护成本。

hugetlbfs、THP 和复合页分别处理不同层面的大页问题。hugetlbfs 提供显式接口和隔离语义，但使用成本较高；THP 对应用透明，却主要围绕固定 2 MiB 粒度工作；复合页在内核内部复用 head/tail 组织方式，但接口容易混入传统 page 假设。Folio 的引入，是为了让页缓存和内存管理代码用更明确的类型表示一个逻辑页，并支撑后续多尺寸页缓存对象。

2.1.2 Folio 在内核中的演进现状与趋势

为了解决上述 hugetlbfs/THP/复合页各自为政以及接口语义混乱的问题，Matthew Wilcox 自 5.15 版本开始在内核中引入了 `struct folio`[11]。Folio 在类型系统上强制区分了“单页/复合页整体”与“页内的子页”。一个 Folio 结构体自身即代表一个完整的、物理连续的内存单元，不再需要区分头尾。

```
// File: include/linux/mm_types.h

struct folio {
    /* public: */
    unsigned long flags;
    struct list_head lru;
    struct address_space *mapping;
    pgoff_t index;
    void *private;
    atomic_t _mapcount;
    atomic_t _refcount;
#ifdef CONFIG_64BIT
    unsigned int _folio_nr_pages;
#endif
    /* ... */
};

// File: include/linux/mm.h & include/linux/pagemap.h

#define page_folio(p) (_Generic((p), \
    const struct page *: (const struct folio *)_compound_head(p), \
    struct page *: (struct folio *)_compound_head(p)))
```



```
#define nth_page(page, n) ((page) + (n))
#define folio_page(folio, n) nth_page(&(folio)->page, n)

void folio_lock(struct folio *folio);
void folio_unlock(struct folio *folio);
void folio_mark_dirty(struct folio *folio);
struct page *folio_file_page(struct folio *folio, pgoff_t index);

void mapping_set_large_folios(struct address_space *mapping);
```

随后，内核新增 `mapping_set_folio_min_order()` 和 `mapping_set_folio_orders()` 等接口。文件系统可以借助这些接口限定页缓存可用的 `folio` 尺寸范围[16][18]。

表 2-1 Folio 在 Linux 内核中的演进时间线

内核版本	发布时间 (约)	关键改动	涉及子系统	说明
5.16	2022 年初	引入 <code>struct folio</code> 及基础 Memory Folios 框架[1][2]	内存管理 / 页缓存	为替换 <code>struct page</code> 奠定基础，尚以 <code>order-0</code> 为主
5.18	2022 年中	提供 <code>mapping_set_large_folios()</code> 接口，文件系统可声明支持 Large Folios[3][4]	页缓存 / 文件系统	<code>page cache</code> 可以按更大粒度分配缓存页
5.19–6.5	2022–2023	大量子系统改造为使用 <code>folio</code> ， <code>page cache</code> 文档明确以 <code>folio</code> 为基本管理单元[5]	页缓存 / KVM / 文件系统等	减少 <code>compound_head()</code> 调用和 <code>head/tail</code> 语义混乱
6.6 及以后	2023–2024	围绕 Large Block Size 与最小 <code>folio order</code> 的 API 演进（ <code>mapping_set_folio_min_order / orders</code> 等）[3][4]	页缓存 / 文件系统	为块大小大于页面大小的系统提供支持，通过改进大块顺序 I/O 提升性能。
6.10 及以后	2024 年及以后	更进一步优化支持 16KB、64KB 等大页，内核支持扩展到更大规模的 <code>memory folio</code> 优化[6]	页缓存 / 文件系统	为高性能 SSD 和服务端级大内存环境提供优化，进一步减少 I/O 延迟与写放大

2.2 iomap 框架：现代化的 I/O 路径

内核文档指出，`iomap` 在 `buffered I/O` 路径中承担了大部分 `folio` 和 页缓存 管理工作[16]。

2.2.1 iomap 数据结构与接口定义

iomap 的核心思想是将文件系统的逻辑块映射结果抽象为一个通用的描述符。无论底层文件系统如何组织数据（Ext4 的 extent 树、XFS 的 B+ 树，或 F2FS 的 NAT 表），向上层返回的都是统一的 struct iomap 结构。在此基础上，iomap 又通过迭代器和一组回调接口，将缓冲 I/O、写回和直接 I/O 均统一在同一套抽象之下[16]。

```
// File: include/linux/iomap.h

struct iomap {
    u64 addr;
    loff_t offset;
    u64 length;
    u16 type;
    u16 flags;
    struct block_device *bdev;
    struct dax_device *dax_dev;
    void *inline_data;
    void *private;
    const struct iomap_folio_ops *folio_ops;
};
```

struct iomap 的每一个实例描述了文件逻辑区间到物理存储区间的一段连续映射：offset/length 表示文件中的字节范围，addr 和 bdev/dax_dev 指定了底层设备与物理偏移，type 用于区分普通映射、空洞（HOLE）、延迟分配（DELALLOC）、未初始化区间等不同语义，private 和 folio_ops 则为文件系统预留了扩展点，可以挂接自定义的 per-folio 状态或校验逻辑[16]。

这些映射并不是孤立使用的，而是通过 struct iomap_iter 串联起来。iomap_iter 保存当前 inode、待处理文件偏移 pos、长度 len、已处理字节数 processed，以及可复用的 struct iomap。通用迭代入口 iomap_iter() 根据文件系统提供的 iomap_begin 回调，持续产出覆盖后续文件区间的 iomap 片段。这一过程会持续到整个 I/O 请求范围被消费完毕[16]。

文件系统通过 struct iomap_ops 向 iomap 注册最核心的两个回调：->iomap_begin 负责在给定文件偏移和长度下填充 struct iomap。->iomap_end 则在 I/O 结束时执行收尾工作（例如更新配额、统计信息或 unwritten extent 转换）[16]。在此之上，iomap 针对不同的 I/O 场景又定义了多组辅助接口，例如 iomap_readahead / iomap_read_folio 负责缓冲读。iomap_file_buffered_write 负责缓冲写入和延迟分配。iomap_writepages 加

载 `struct iomap_writeback_ops` 完成脏页写回。而 `iomap_dio_rw` 则统一了 `O_DIRECT` 的读写路径[16]。

在页缓存这一层面，`iomap` 还提供了 `struct iomap_folio_ops` 来定制 `per-folio` 行为。内核文档给出的定义如下：

```
// File: include/linux/iomap.h / fs/iomap/buffered-io.c
```

```
struct iomap_folio_ops {
    struct folio *(*get_folio)(struct iomap_iter *iter,
                               loff_t pos, unsigned len);
    void (*put_folio)(struct inode *inode, loff_t pos,
                     unsigned copied, struct folio *folio);
    bool (*iomap_valid)(struct inode *inode,
                      const struct iomap *iomap);
};
```

其中 `get_folio/put_folio` 允许文件系统在写入路径上为每个 `folio` 设置和提交自定义的状态。而 `iomap_valid` 则用于应对空间映射在并发 I/O 过程中发生变化的情况：文件系统可以在 `->iomap_begin` 中记录一个“有效性 cookie”。在 `iomap_valid` 中重新检查当前映射是否仍然成立，如果失效则重新取样映射[16]。通过这一机制，`iomap` 在不强制文件系统持有重锁的前提下，仍然能够在复杂并发场景下保持映射与页缓存状态的一致性。

内核文档指出，`per-folio` 状态对象比 `buffer_head` 更轻量，因为每个 `folio` 只需要一个状态对象[16]。

在更高层次上，`iomap` 通过统一 `buffered I/O` 和 `direct I/O` 的映射逻辑。为大块顺序 I/O 和多种存储介质（包括 NVMe SSD、直连持久内存等）提供了统一的抽象。对于像 F2FS 这样希望利用 `Large Folios` 和大粒度 I/O 的日志结构文件系统来说。`iomap` 消除了对 `buffer_head` 的依赖，还提供了丰富的钩子（如 `iomap_writeback_ops` 和 `iomap_dio_ops`）。便于在大页写回和复杂的空间管理策略之间进行解耦和复用[16][17]。

2.2.2 迭代器模式与区间映射优势

传统 `get_block` 接口每次只能映射一个文件系统块（通常 4KB），处理大文件时需要循环调用数百万次。而 `iomap` 引入了 `iomap_iter` 迭代器：

```
// File: fs/iomap/buffered-io.c

while (iomap_iter(&iter, ops) > 0) {
    iter.processed = iomap_copy_actor(...);
}
```

这种区间化的处理方式天然契合 Large Folios。当文件系统返回一个覆盖 2MB 范围的 iomap 时。内核可以一次性分配并处理一个 2MB 的 Huge Folio，或者批量处理多个 Large Folios，从而将 I/O 路径上的 CPU 开销降低数个数量级。内核上游正在将 iomap 变成所有文件系统统一 I/O 接口。目前正往各个主流文件系统逐步迁移。XFS 率先完成全 IO 路径的 iomap 支持，ext4 文件系统也在逐步推进，下表列出目前内核主流文件系统的 iomap 支持情况：

表 2-2 iomap 框架在主流 Linux 文件系统中的应用现状

文件系统	采用 iomap 的场景	典型内核版本	Large Folio / 大块 I/O 支持	备注
XFS	数据路径全面使用 iomap：缓冲 I/O、直接 I/O、DAX、fiemap、SEEK_DATA/SEEK_HOLE 等	4.x 即开始使用，5.x 以后成为 iomap 主要示例，6.x 中作为参考实现长期维护	完整支持 Large Folios，页缓存层基本只操作 folio 而不直接处理 struct page	iomap 的最早用户之一，也是当前特性最完整、经验最成熟的文件系统
Btrfs	目前主要在 direct I/O 路径使用 iomap，缓冲 I/O 仍保留 Btrfs 自己的复杂逻辑	Linux 5.8 起在 direct I/O 路径接入 iomap	direct I/O 绕过页缓存，对 Large Folio 的依赖相对较弱；后续计划逐步改造缓冲 I/O	内核文档与开发者 Q&A 中明确：当前仅少量 IOMAP 调用，主要用来处理 direct I/O
Ext4	direct I/O 已切换到 iomap；regular file 的缓冲 I/O 正在分阶段从 buffer_head 迁移到 iomap	direct I/O：5.5 起；缓冲 I/O：6.9 左右开始有较大规模补丁，部分发行版在评估启用	基于 iomap 的缓冲 I/O 路径设计为支持 Large Folios，大页写回依赖 per-fsblock 脏位图	当前 ext4 同时保留 buffer_head 路径，在启用某些特性（如 data=journal 等）时会回退
Ext2	direct I/O 使用 iomap；缓冲 I/O 转换工作仍在推进中	Linux 6.6 起增加 direct I/O 的 iomap 支持	Large Folio 相关行为主要由 iomap 核心负责，ext2 作为简单文件系统用于演练迁移路径	在 LSFMM 会议报告中经常被用作示例，用于说明从 get_block 接口迁移到 iomap 的步骤
F2FS	direct I/O 路径已经通过 iomap_dio_rw 等接口集成 iomap；缓冲读写 + Large Folio 支持有专门 RFC 补	direct I/O 支持早于 large folio；2024 年起陆续提	目标是在缓冲读写路径上利用 Large Folios 和 per-fsblock	当前上游仍在 review / 演进中，你的课题工作正是顺着这条

	丁集	交“基于 F2FS 扩展 iomap 的 large folio 缓冲 I/O”补丁	脏位图，降低写放大并优化顺序 I/O	路线解决 F2FS 上的工程落地问题
GFS2	在集群文件系统的 direct I/O 路径使用 iomap_dio_rw 等接口，需要与 glock 锁机制配合	5.x 期间引入，对 mmap + page fault 与 glock 的交互有专门修复补丁	Large Folio 支持更多依赖 iomap 核心能力，本身主要用于验证集群场景下的并发与恢复语义	邮件列表讨论中多次提到：需要小心处理禁用 page fault 与 iomap_dio_rw 的配合，以避免死锁
Zonefs	作为“分区即文件”的简单文件系统，数据路径几乎完全依赖 iomap 提供的大块顺序 I/O 抽象	随 ZNS SSD 支持在 5.x 内核中加入，并持续与 iomap 一起演进	天然适合大块、顺序的 I/O 模式，与 Large Folio 和大块写入高度契合	常被内核文档用作“简单文件系统”示例，用来说明在无复杂特性的前提下如何完整迁移到 iomap

第三章 F2FS 支持 Large Folios 的核心难点

F2FS 接入 Large Folios 后，问题首先出现在粒度不一致上。页缓存以 folio 为单位管理内存，而 F2FS 在本文实验配置下仍围绕文件系统 block、压缩簇和 segment 维护状态。两侧粒度并不相同，后续实现需要把页缓存子范围与 F2FS 的块映射、压缩簇和 GC 搬运范围对应起来。

3.1 块映射与 I/O 提交的粒度失配

3.1.1 逐页处理带来的 CPU 开销

F2FS 原有 buffered read 路径主要围绕单个 page 展开。以 f2fs_read_single_page 为例，一次调用只处理一个 4 KiB 页面；当页缓存分配出更大的 folio 时，同一 folio 内的多个 4 KiB 子范围仍可能重复进入 dnode 查找、锁获取和块映射流程。源码位置：fs/f2fs/data.c:2196, 2885-2889。

这类重复工作会削弱 Large Folio 的收益。页缓存虽然一次覆盖了更大的内存范围，但文件系统侧如果仍逐页完成映射和提交，CPU 时间仍会消耗在多次元数据访问和状态更新上。

3.1.2 Large Folio 子区间多 BIO 提交与 I/O 生命周期失配

传统 buffered read 路径中，I/O 完成通常按 page 收尾。一个 page 对应的读取完成后，路径即可设置 uptodate 并释放锁。Large Folio 改变了这个假设：一个 folio 内

可能有多个磁盘子区间，多个 bio 的完成顺序也不固定。如果某个 bio 先完成就结束整个 folio，folio 锁语义会早于真实数据状态。

以 16 KiB folio 为例，读取过程可能被拆成两个 8 KiB bio。第一个 bio 完成时，第二个 bio 可能仍在队列中；此时只能标记对应子区间，不能结束整个 folio。反过来，如果完全不记录子区间完成情况，又无法保留已经读好的范围。

因此，读取路径需要同时记录两类信息：当前 iomap 区间覆盖哪些文件字节，以及当前 folio 内还有多少字节尚未完成读取。read_bytes_pending 就承担第二类状态，它把多个 bio 的完成结果重新汇总到同一个 folio 上。

```
// File: fs/iomap/buffered-io.c

/* iomap_readahead & iomap_readahead_iter */
while (iomap_iter(&iter, ops) > 0)
    iter.status = iomap_readahead_iter(&iter, &ctx);

static int iomap_readahead_iter(struct iomap_iter *iter,
                                struct iomap_readpage_ctx *ctx)
{
    while (iomap_length(iter)) {
        /* ... */
        ctx->cur_folio = readahead_folio(ctx->rac);
        ret = iomap_readpage_iter(iter, ctx);
        /* ... */
    }
}
```

在 iomap_readpage_iter 内部，read_bytes_pending 表示当前 folio 中仍处于读取过程的字节数。代码在把子区间加入 bio 之前先增加计数，在 I/O 完成回调中再按同一范围递减。只有计数归零时，路径才调用 folio_end_read 结束该 folio 的读取流程。

```
// File: fs/iomap/buffered-io.c

ctx->cur_folio_in_bio = true;
if (ifs) {
    spin_lock_irq(&ifs->state_lock);
    ifs->read_bytes_pending += plen;
    spin_unlock_irq(&ifs->state_lock);
}
```

```

bio_add_folio_nofail(ctx->bio, folio, plen, poff);

static void iomap_read_end_io(struct bio *bio)
{
    int error = blk_status_to_errno(bio->bi_status);
    struct folio_iter fi;

    bio_for_each_folio_all(fi, bio)
        iomap_finish_folio_read(fi.folio, fi.offset, fi.length, error);
    bio_put(bio);
}

static void iomap_finish_folio_read(struct folio *folio, size_t off,
                                   size_t len, int error)
{
    struct iomap_folio_state *ifs = folio->private;
    bool uptodate = !error;
    bool finished = true;

    if (ifs) {
        unsigned long flags;

        spin_lock_irqsave(&ifs->state_lock, flags);
        if (!error)
            uptodate = ifs_set_range_uptodate(folio, ifs, off, len);

        ifs->read_bytes_pending -= len;
        finished = !ifs->read_bytes_pending;
        spin_unlock_irqrestore(&ifs->state_lock, flags);
    }

    if (finished)
        folio_end_read(folio, uptodate);
}

```

Read_bytes_pending 示意图

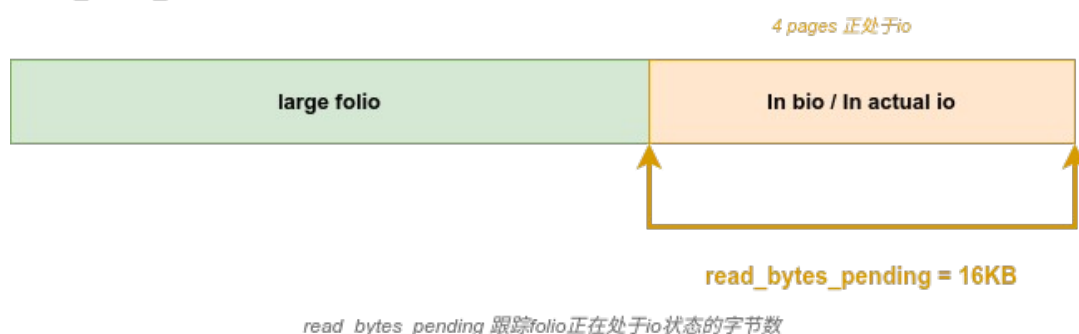


图 3-2 I/O 生命周期图

这段逻辑的约束在于顺序：先增加 `read_bytes_pending`，再把子区间加入 `bio`。即使块层很快完成该 `bio`，完成回调看到的计数也已经包含本次提交范围，不会出现 `bio` 已完成但 `folio` 待读计数尚未记录的中间状态。

仍以 16 KiB `folio` 和两个 8 KiB `bio` 为例。第一个 `bio` 提交前，计数增加到 8192；第二个 `bio` 提交前，计数增加到 16384。若第一个 `bio` 先完成，计数只下降到 8192，`folio` 仍保持未完成状态；第二个 `bio` 完成后，计数归零，`folio` 才能解锁。这样完成顺序可以变化，但 `folio` 的结束条件始终绑定到全部相关子区间。

3.1.3 空洞文件与 I/O 区间划分的一致性

`f2fs_map_blocks` 需要区分 `dnode` 是否已经分配、目标块是否为空洞，以及调用者当前使用的映射模式。这些模式包括 `PRECACHE`、`IOMAP` 等，分别服务于不同的读写路径。

空洞文件还会影响 `iomap` 迭代边界。`dnode` 已经存在但逻辑块为空洞时，F2FS 不能简单把当前 `folio` 全部视为普通映射；`IOMAP` 模式下需要更新 `m_next_pgofs`，让下一次迭代从空洞之后继续。

```
// File: fs/f2fs/data.c

} else if (is_hole) {
    switch (flag) {
        case F2FS_GET_BLOCK_PRECACHE:
            goto sync_out;
        case F2FS_GET_BLOCK_IOMAP:
            if (map->m_next_pgofs)
                *map->m_next_pgofs = pgofs + 1;
            break;
    }
}
```

```
}  
map->m_pblk = blkaddr;
```

在 4 KiB page 模型下，遇到空洞后结束本次映射通常只影响一个 page。Large Folio 下，一个 folio 覆盖多个逻辑页，iomap_iter 又倾向于返回连续区间；如果空洞边界处理不清，映射可能被频繁打断，也可能跳过本来可以继续处理的范围。长空洞场景还会放大这种边界问题，导致页缓存分配和零填充范围不必要地扩大。

因此，迁移到 iomap 的 f2fs_map_blocks 路径需要在区间层面区分真实映射和空洞。该区分不是格式问题，而是决定后续 folio 内哪些范围读取磁盘、哪些范围填零，以及哪些范围可以继续合并提交。

3.2 关键数据结构的冲突与兼容

3.2.1 私有字段复用带来的状态冲突

folio->private 是页缓存对象留给文件系统使用的私有字段。F2FS 原有路径会在这个字段中保存 atomic write、GC migration 等私有标志；这些标志服务于 F2FS 自己的写入、压缩和搬运路径。

iomap buffered I/O 也需要 folio->private。它通常把该字段解释为 iomap_folio_state 指针，用于保存 folio 内的 uptodate、dirty 等位图状态。于是，同一个字段在不同路径中出现两种解释：F2FS 标志位，或 iomap 状态指针。

这一冲突集中体现在 folio->private 的所有权上。F2FS 需要保存 GC、压缩、atomic 等私有标志，iomap 则需要保存指向 iomap_folio_state 的状态指针。压缩写回还需要额外的 pending 计数，用于延迟解锁和完成判断。因此，适配方案不能简单保留某一方的解释方式，而需要把文件系统私有标志、iomap 位图和 pending 计数合并到统一的状态对象中。

3.2.2 状态误判导致的崩溃风险

如果不统一这个字段，崩溃风险很直接：F2FS 写入的是小整数标志位，iomap 读取时却可能按指针解引用。此时 private 的值不再指向有效状态对象，后续位图访问就可能落到非法地址。

因此，接入统一状态结构后，旧的 set_page_private/get_page_private 调用不能继续散落在各条路径中。普通数据 folio、GC 搬运 folio 和压缩路径都需要通过同一组访问接口读取私有标志，避免再次绕回直接解释 private 的写法。

例如 `f2fs_release_folio` 和 `f2fs_is_compressed_folio`。它们属于 F2FS 代码，但会被通用页缓存回收路径调用；一旦 `private` 语义不一致，问题会从文件系统内部传到 VFS 和内存回收路径。调用栈如下：

```
Call trace:
show_stack+0x18/0x24 (C)
dump_stack_lvl+0x78/0x90
dump_stack+0x18/0x24
  f2fs_release_folio+0x18/0x50
filemap_release_folio+0x78/0xbc
mapping_evict_folio+0x8c/0xac
mapping_try_invalidate+0xbc/0x1a0
invalidate_mapping_pages+0x14/0x20
drop_pagecache_sb+0xa4/0x128
iterate_supers+0x88/0x100
drop_caches_sysctl_handler+0x7c/0x190
proc_sys_call_handler+0x17c/0x258
proc_sys_write+0x14/0x20
vfs_write+0x2b0/0x35c
ksys_write+0x68/0xfc
__arm64_sys_write+0x1c/0x28
invoke_syscall+0x48/0x110
el0_svc_common.constprop.0+0x40/0xe8
do_el0_svc+0x20/0x2c
el0_svc+0x30/0xd8
el0t_64_sync_handler+0x144/0x168
el0t_64_sync+0x198/0x19c
```

该调用栈说明，`folio->private` 的语义会经过 `filemap_release_folio`、`mapping_evict_folio` 等通用页缓存路径继续传播。`drop_caches` 等用户可见接口也可能触发这些路径，因此状态访问接口必须在 F2FS 和通用页缓存之间保持一致。

`order-0 folio` 也不能例外。它大小上仍等同传统 `page`，但仍可能进入 `iomap buffered write`、GC 搬运和页缓存回收路径。只要这些路径共享 `private` 字段，就必须使用同一种状态对象。

GC 代码路径中的 `set_page_private_gcing` 调用如下所示：

```
// File: fs/f2fs/gc.c

/* move_data_page in gc.c */
static int move_data_page(struct inode *inode, block_t bidx, int gc_type,
                          unsigned int segno, int off)
{
    struct folio *folio;
    int err = 0;

    // ...
    folio = f2fs_get_lock_data_folio(inode, bidx, true);
    // ...

    if (gc_type == BG_GC) {
        // ...
        folio_mark_dirty(folio);

        set_page_private_gcing(&folio->page);
    } else {
        // ...
        set_page_private_gcing(&folio->page);
        err = f2fs_do_write_data_page(&folio);
        // ...
    }

out:
    f2fs_folio_put(folio, true);
    return err;
}
```

触发过程可以按路径理解：GC 在 `move_data_page` 中处理 `order-0 folio`，并把 `ONGOING_MIGRATION` 等标志写入 `folio->private`；同一文件随后进入 `iomap buffered write` 时，`iomap` 发现 `private` 非空，可能把它当成状态指针使用。两条路径对同一字段的解释不同，状态误判由此产生。

因此，统一状态结构不能只覆盖高阶 `folio`。凡是可能进入 `iomap` 的 `folio`，包括 `order-0 folio`，都需要用同一种对象保存 F2FS 私有标志和 `iomap` 位图。这样才能让 GC、buffered write 和页缓存回收路径看到同一种 `private` 语义。

3.2.3 I/O 描述粒度限制

F2FS 使用 `f2fs_io_info` 描述一次 I/O 请求。原结构只携带 `page` 指针时，提交路径很难表达“当前 `page` 位于哪个 `folio`、对应 `folio` 内哪个偏移”。Large Folio 写回需要这个偏移信息，否则 `bio` 中加入的范围可能从 `folio` 起始位置计算，造成提交范围与实际逻辑块不一致。源码位置：`fs/f2fs/f2fs.h:1230-1255`。

3.3 压缩文件场景下的复杂性

3.3.1 压缩簇与 Folio 边界的不对齐

F2FS 压缩文件读取不能简单复用普通 `buffered read`。普通文件可以把磁盘块直接读入目标 `folio`；压缩文件则需要先读取压缩数据，再解压到目标 `folio`。后续状态管理必须同时覆盖磁盘 I/O 和解压完成两个阶段。

压缩读路径先把压缩数据读入临时缓冲页，再由解压逻辑把明文写入目标 `folio`。目标 `folio` 的 `uptodate` 状态不能只跟随磁盘 `bio` 完成，还要等解压结果真正落入对应字节范围。

F2FS 还支持部分压缩，同一文件内可能同时存在压缩簇和未压缩块。Large Folio 使这个问题更明显：一个 `folio` 可能跨过多个压缩簇，也可能同时覆盖非压缩数据。这样，`folio` 边界、压缩簇边界和磁盘映射边界不再重合，读取路径必须按子区间选择直接读或解压读。

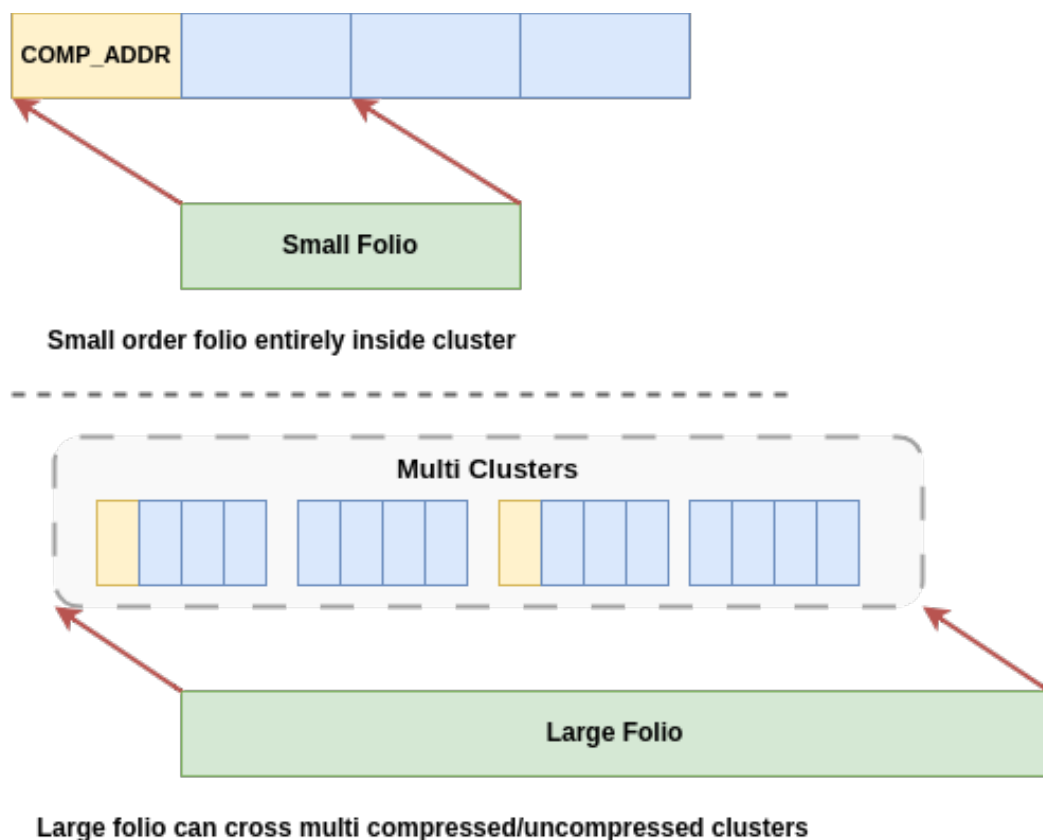


图 3-3 压缩簇与 Folio 边界错位示意图

3.3.2 混合 I/O 模型（压缩与非压缩混排）

核心挑战来自混合 I/O 模型和状态管理。一个 Large Folio 的不同部分可能由不同路径填充。非压缩子区间可以通过 `iomap_readpage_iter` 走“磁盘到 Folio”的标准读取路径。

直接读和压缩读的完成时机不同。非压缩数据在存储 I/O 完成后即可更新对应范围，压缩数据要等解压结束后才写入目标 folio。若没有统一的 pending 计数，直接读部分可能提前推动 folio 解锁，而压缩部分仍未完成。

因此，压缩 buffered read 的核心不是“多一次解压”，而是 folio 生命周期如何覆盖多种子区间来源。实现既要避免提前解锁，也要保留直接读和解压读之间的并行空间。

3.4 脏页写回中的写放大风险

3.4.1 F2FS 原有写回模型与写放大根源

在引入 `iomap` 之前，F2FS 对脏页的管理主要以 `page` 为单位。只要 folio 中任意 4 KiB 子页被修改，相关 `page` 就会被标记为脏。迁移到 Large Folio 后，如果仍沿用

这种粗粒度语义，一个子页变脏就可能导致整个 folio 被视为待写回范围，从而放大实际 I/O。

在 4 KiB page 时代，按 page 标记 dirty 与提交范围基本一致。Large Folio 下，一个应用只修改 4 KiB 子范围，也可能让更大的 folio 进入写回流程。对日志结构的 F2FS 来说，额外提交的范围会转化为新的追加写入，从而增加日志空间占用和后续 GC 压力。

F2FS 的异地写入语义还会把这种影响累积起来。同一逻辑位置多次小幅修改时，物理位置会不断追加新块，旧块等待后续 GC 回收。如果 dirty 状态只能表示“整个 folio 需要写回”，写回路径就无法区分真正修改的子范围。

iomap_folio_state 在 folio 内维护逐块状态位，用于记录每个文件系统块是否 uptodate、是否 dirty。借助这一结构，写回路径可以定位到 folio 内实际变脏的块，而不是默认提交整个 folio。对于带有压缩簇和日志结构写入特性的 F2FS，还需要把这一位图语义与文件系统私有状态合并使用。

3.4.2 Large Folio 与压缩写回的架构性挑战

压缩文件还引入另一套粒度。F2FS 以压缩簇组织压缩数据，Large Folio 的大小则由页缓存分配结果决定。两者没有固定包含关系：一个 folio 可能覆盖多个簇，一个簇也可能只占据 folio 的一部分。

这种关系直接影响 dirty 追踪。写回路径不能只问“folio 是否 dirty”，还要知道 dirty 范围落在哪些压缩簇内。逐块 dirty 位图需要继续传给压缩写回路径，让后者只围绕受影响的簇组织提交。

生命周期也不再只有一个层级。folio 的锁和 writeback 标志作用于整个内存对象，而不同压缩簇可能处于排队、压缩、提交或失效等不同阶段。若仍用一次 folio 级完成动作概括这些状态，就很难判断哪个子范围已经安全写出。

并发截断会进一步放大边界问题。旧重试逻辑遇到 -EAGAIN 后倾向于重新锁定 folio 并从较粗的范围重试；在 Large Folio 和压缩簇同时存在时，重试期间某些子范围可能已经被截断、重写或重新映射。后续写回必须重新确认失败子范围，而不是默认复用旧的整 folio 状态。

因此，-EAGAIN 后的重试也要保留字节范围。失败的是某个 folio 子范围，重试入口就应围绕该范围重新检查映射和状态；如果重新提交整个 folio，压缩簇和物理块分配都可能被重复触发。

本节得到的结论是：写回路径需要同时解决三个边界问题。第一，dirty 状态必须落到 folio 内子范围；第二，压缩簇与 folio 的范围关系需要显式维护；第三，重试和截断不能丢失失败子范围。第四章的写回设计即围绕这三点展开。

第四章 F2FS Large Folio 缓冲 I/O 设计与实现

4.1 设计目标与总体思路

第三章已经从框架层面分析了 Large Folio 与 iomap 的关系。本章进一步讨论 F2FS 中的具体实现。F2FS 原有 buffered I/O 路径长期围绕 page 粒度组织，块映射、页缓存状态、脏页写回和 GC 搬运都带有较强的单页假设。引入 Large Folio 后，一个 folio 内可能同时包含多个文件系统块，原有路径中的单页状态和单块提交方式不能直接覆盖新的内存粒度。

本章的实现目标可以概括为两点：第一，在普通文件读写路径中建立 F2FS 块级映射结果到 iomap 字节区间之间的转换。使页缓存侧能够围绕较大的连续区间组织 buffered I/O；第二，在 F2FS 特有路径中补充 Large Folio 所需的状态管理。覆盖读完成计数、folio 内脏区记录、writeback 子范围提交、GC 迁移标志以及压缩文件的簇级处理。

本章按实现依赖关系展开。4.2 节先说明普通文件路径中 f2fs_map_blocks 到 f2fs_set_iomap 的转换过程，这是后续 read/write 接入 iomap 的基础。4.3 节说明 f2fs_folio_state 如何承载 F2FS 私有状态和 iomap 状态。4.4 节讨论脏页写回中 folio 内脏区定位和子范围提交。4.5 节讨论 GC 路径在 Large Folio 下的适配。4.6 节讨论压缩文件中的簇级读取、部分簇补齐和延迟解锁处理。

4.2 基于区间映射的普通文件 I/O 路径

本节讨论普通文件在 buffered I/O 场景下的 iomap 化过程。这里的“普通文件”指不经过压缩簇特殊处理的文件路径。其读写流程主要围绕 f2fs_map_blocks、f2fs_set_iomap 和 iomap buffered I/O 回调展开。

在第三章的框架分析基础上，本节进一步说明 F2FS 如何把自身的块级映射结果整理为 iomap 所需的字节区间，并在普通文件的读取和写入路径中使用该转换结果。

4.2.1 F2FS 块映射与 iomap 字节区间的粒度差异

F2FS 内部的块映射结果主要由 `struct f2fs_map_blocks` 表示，其中 `m_lblk` 表示文件内逻辑块号，`m_pblk` 表示物理块号。`m_len` 表示连续块长度，`m_flags` 表示该区间的状态。该结构适合 F2FS 的 `dnode` 和 `extent cache` 逻辑，但 `iomap` 框架并不直接使用块号。`iomap` 需要的是文件逻辑字节偏移、区间长度、块设备、磁盘字节地址以及 `IOMAP_MAPPED`、`IOMAP_HOLE`、`IOMAP_UNWRITTEN` 等通用状态。

两类结构的差异可以概括为两层语义。F2FS 负责回答从哪个逻辑块开始、连续多少块、对应哪个物理块。`iomap` 负责回答从哪个文件字节开始、连续多少字节、应该按何种类型处理。因此，普通文件路径的关键不是简单替换读写函数，而是在 F2FS 块映射和 `iomap` 字节区间之间建立稳定的转换层。

在具体实现中，这个转换层分别服务于普通读取和普通写入的 `iomap_begin` 回调。前者只查询已有块映射，后者还需要处理空洞区间的预分配。后续的 `writeback` 和 `GC` 仍需要处理 F2FS 的特殊状态，但普通读写的入口可以先统一到同一套区间映射语义上。

4.2.2 块区间到 iomap 区间的转换

`f2fs_set_iomap` 是普通文件路径中的核心转换函数。F2FS 在 `f2fs_map_blocks` 中得到的是文件系统块级结果，而 `iomap` 框架要求文件系统返回字节区间。两者的语义并不相同：`f2fs_map_blocks` 描述逻辑块号、物理块号和连续块数；`struct iomap` 描述文件字节偏移、磁盘字节地址、区间长度和区间类型。因此，该函数不是普通的赋值辅助函数，而是普通 `buffered read/write` 能够接入 `iomap` 的边界转换层。

这一转换具有明确的对比关系：

- `map->m_lblk` 和 `map->m_pblk` 仍以 F2FS 块为单位，`iomap->offset` 和 `iomap->addr` 必须转换为字节地址。
- `map->m_len` 表示连续文件系统块数，`iomap->length` 表示连续字节长度。

`NULL_ADDR`、`NEW_ADDR` 和 `F2FS_MAP_MAPPED` 是 F2FS 内部状态。`iomap` 侧需要表达为 `IOMAP_HOLE`、`IOMAP_UNWRITTEN` 或 `IOMAP_MAPPED`。

- `F2FS_MAP_NODNODE` 表示 `dnode` 缺失导致的空洞，转换时需要借助 `m_next_pgofs` 计算空洞边界，而不能简单返回单块长度。

```
/* Source: fs/f2fs/data.c:5086-5137, f2fs_set_iomap */
int f2fs_set_iomap(struct inode *inode, struct f2fs_map_blocks *map,
                  struct iomap *iomap, unsigned int flags,
                  loff_t offset, loff_t length, bool dio)
{
```

```

iomap->offset = F2FS_BLK_TO_BYTES(map->m_lblk);

if (map->m_flags & F2FS_MAP_MAPPED) {
    if (dio && WARN_ON_ONCE(map->m_pblk == NEW_ADDR))
        return -EINVAL;

    iomap->length = F2FS_BLK_TO_BYTES(map->m_len);
    iomap->bdev = map->m_bdev;

    if (map->m_pblk != NEW_ADDR) {
        iomap->type = IOMAP_MAPPED;
        iomap->flags |= IOMAP_F_MERGED;
        iomap->addr = F2FS_BLK_TO_BYTES(map->m_pblk);
    } else {
        iomap->type = IOMAP_UNWRITTEN;
        iomap->addr = IOMAP_NULL_ADDR;
    }
} else if (map->m_pblk == NULL_ADDR) {
    if (map->m_flags & F2FS_MAP_NODNODE)
        iomap->length =
            F2FS_BLK_TO_BYTES(*map->m_next_pgofs) -
            iomap->offset;
    else
        iomap->length = F2FS_BLK_TO_BYTES(map->m_len);
    iomap->type = IOMAP_HOLE;
    iomap->addr = IOMAP_NULL_ADDR;
} else if (map->m_pblk == NEW_ADDR) {
    iomap->length = F2FS_BLK_TO_BYTES(map->m_len);
    iomap->type = IOMAP_UNWRITTEN;
    iomap->addr = IOMAP_NULL_ADDR;
}

if (map->m_flags & F2FS_MAP_NEW)
    iomap->flags |= IOMAP_F_NEW;
if ((inode->i_state & I_DIRTY_DATASYNC) ||
    offset + length > i_size_read(inode))
    iomap->flags |= IOMAP_F_DIRTY;

return 0;
}

```

这段代码的第一步是设置 `iomap->offset`。由于 `map->m_lblk` 仍是文件系统块号，代码通过 `F2FS_BLK_TO_BYTES` 将其转换为文件逻辑字节偏移。后续的 `iomap->length` 和 `iomap->addr` 也采用同样的转换方式，因此 `iomap` 层看到的是字节区间，而不是 `F2FS` 内部的块编号。

当 `map->m_flags` 包含 `F2FS_MAP_MAPPED` 时，说明该区间已经存在有效映射。如果 `m_pblk` 不是 `NEW_ADDR`，函数把该区间标记为 `IOMAP_MAPPED`，并将

物理块号转换为磁盘字节地址。这里设置 IOMAP_F_MERGED 的含义是该映射已经由 F2FS 内部合并为连续区间，上层可以按区间处理。

hole 场景需要区分两种情况。一种是 dnode 已存在但目标块为空洞，此时长度可以直接来自 map->m_len；另一种是 dnode 本身不存在。代码通过 F2FS_MAP_NODNODE 识别该情况，并根据 m_next_pgofs 计算 hole 的结束位置。这样，iomap 得到的 hole 不局限于单个块。而是覆盖当前可以确认的连续空洞范围。

从普通文件路径看，f2fs_set_iomap 承担了三项集中职责：第一，把 F2FS 的块号空间转换为 iomap 的字节地址空间；第二。把 F2FS 的块状态转换为 iomap 的通用区间类型；第三，把新分配、脏数据和无 dnode 空洞等 F2FS 语义保留下来。避免上层 iomap 迭代过程丢失文件系统语义。后续普通读路径和普通写路径都以该函数作为映射返回点，因此它是第四章普通文件 I/O 路径中最主要的代码。

4.2.3 普通文件读取路径

普通文件的 buffered read 入口不直接创建块，也不处理压缩簇。它根据读请求的字节范围计算逻辑块范围，调用 f2fs_map_blocks 获取映射，再通过 f2fs_set_iomap 返回给 iomap 框架。

```
/* Source: fs/f2fs/data.c:5145-5174, f2fs_buffered_read_iomap_begin */
static int
f2fs_buffered_read_iomap_begin(struct inode *inode, loff_t offset,
                               loff_t length, unsigned int flags,
                               struct iomap *iomap, struct iomap *srcmap)
{
    pgoff_t next_pgofs = 0;
    struct f2fs_map_blocks map = {};
    int err;

    map.m_lblk = F2FS_BYTES_TO_BLK(offset);
    map.m_len = F2FS_BYTES_TO_BLK(offset + length - 1) -
                map.m_lblk + 1;
    map.m_next_pgofs = &next_pgofs;
    map.m_may_create = false;

    err = f2fs_map_blocks(inode, &map, F2FS_GET_BLOCK_IOMAP);
    if (err)
        return err;
    if (WARN_ON_ONCE(map.m_pblk == COMPRESS_ADDR))
        return -EINVAL;

    return f2fs_set_iomap(inode, &map, iomap, flags,
                          offset, length, false);
}
```

这一路径中的 `map.m_may_create = false` 表明读操作只查询已有映射，不会为 hole 创建新块。F2FS_GET_BLOCK_IOMAP 表示本次查询服务于 iomap 区间处理，F2FS 可以返回可合并的连续块范围。若返回结果指向 COMPRESS_ADDR，该路径直接报错，因为压缩文件需要通过压缩簇读路径处理，不能按普通连续块读取。

读取路径的改动集中在映射获取方式上。f2fs_map_blocks 可以利用 extent cache 或 dnode 遍历返回连续映射，f2fs_set_iomap 再把该结果转换为 iomap 区间。这样，普通读路径不需要在入口处反复围绕单个 page 构造块映射，而是把连续区间交给 iomap 迭代过程处理。

4.2.4 普通文件写入路径与空间预分配

普通文件写入路径与读取路径共享 f2fs_set_iomap，但写入前需要处理空洞和预分配问题。写请求到达时，文件系统先把字节范围转换为逻辑块范围，查询已有映射；如果发现目标位置为空洞，则调用预分配接口为该区间准备块映射。

```
/* Source: fs/f2fs/data.c:5935-5954, f2fs_buffered_write_iomap_begin */
block_t start_blk = F2FS_BYTES_TO_BLK(offset);
block_t len_blks = F2FS_BYTES_TO_BLK(offset + length - 1) -
    start_blk + 1;

err = f2fs_map_blocks_iomap(inode, start_blk, len_blks, &map);
if (err && err != -ENOSPC)
    return err;

if (map.m_pblk == NULL_ADDR) {
    err = f2fs_map_blocks_preallocate(inode, map.m_lblk,
        len_blks, &map);
    if (err && err != -ENOSPC)
        return err;
}

if (WARN_ON_ONCE(map.m_pblk == COMPRESS_ADDR))
    return -EIO;

return f2fs_set_iomap(inode, &map, iomap, flags,
    offset, length, false);
```

这段逻辑对应普通文件写入的准备阶段。f2fs_map_blocks_iomap 先尝试在目标区间内找到已有映射；当目标区间为空洞时。f2fs_map_blocks_preallocate 将写入意图传递给 F2FS 的块分配逻辑。与单页写入相比。这里传入的是 start_blk 和 len_blks 所描述的区间，因此连续写入时更容易得到较长的映射结果。

当映射准备完成后，写路径仍通过 f2fs_set_iomap 把结果交给 iomap。后续的用户数据拷贝、folio 准备、脏区标记和写入迭代由 iomap_file_buffered_write 驱动。F2FS 在 file.c 中根据文件类型选择普通 iomap 写入操作或原子文件写入操作。

```

/* Source: fs/f2fs/file.c:4956-4968, f2fs_iomap_buffered_write */
static ssize_t f2fs_iomap_buffered_write(struct kiocb *iocb,
                                         struct iov_iter *i)
{
    struct inode *inode = file_inode(iocb->ki_filp);

    if (f2fs_is_atomic_file(inode))
        return iomap_file_buffered_write(
            iocb, i, &f2fs_buffered_write_atomic_iomap_ops, NULL);

    return iomap_file_buffered_write(
        iocb, i, &f2fs_buffered_write_iomap_ops, NULL);
}

```

普通写路径负责把写入范围映射为 iomap 区间，并为 hole 写入准备必要的块。写入后一个 folio 中哪些字节被修改、哪些子页需要写回、何时可以解锁，仍要依赖后续的 folio state、writeback 和 GC 适配。本章后面的 4.3 至 4.5 正是围绕这些正确性问题展开。

4.3 面向 Large Folio 的状态管理

普通文件路径完成映射转换后，还需要解决 folio 内状态表达问题。F2FS 原有 page 路径大量使用 page->private 保存私有标志，例如 GC 迁移、原子写、inline 数据等。iomap buffered I/O 也会使用 folio 私有状态保存 uptodate 和 dirty 位图。Large Folio 场景下，两类状态不能再简单地各自占用 folio->private，否则会出现所有权冲突。

本节讨论的核心不是单纯增加一个结构体，而是在同一个 folio->private 入口下同时满足三类状态需求。第一。iomap 需要记录 folio 内每个文件系统块的 uptodate 和 dirty 状态；第二。F2FS 需要保留 GC、atomic、inline 等私有标志；第三。Large Folio 的异步读写需要记录尚未完成的字节数。f2fs_folio_state 的设计目的就是要把这三类状态集中到一个对象中。避免不同路径对 folio->private 作出不一致解释。

4.3.1 统一状态结构

f2fs_folio_state 的核心字段如下：

```

/* Source: fs/f2fs/f2fs_ifs.h:13-58, f2fs_folio_state */
#define F2FS_IFS_MAGIC 0xf2f5
#define F2FS_IFS_PRIVATE_LONGS 2

struct f2fs_folio_state {
    spinlock_t state_lock;
    unsigned int read_bytes_pending;
    atomic_t write_bytes_pending;
    unsigned long state[];
};

```

```
static inline size_t f2fs_ifs_iomap_longs(struct folio *folio)
{
    unsigned int nr_blocks = i_blocks_per_folio(folio_inode(folio), folio);

    return BITS_TO_LONGS(2 * nr_blocks);
}

static inline unsigned long *
f2fs_ifs_private_flags_ptr(struct f2fs_folio_state *fifs,
                           struct folio *folio)
{
    return &fifs->state[f2fs_ifs_iomap_longs(folio)];
}

static inline atomic_t *
f2fs_ifs_dirty_bytes_pending_ptr(struct f2fs_folio_state *fifs,
                                 struct folio *folio)
{
    return (atomic_t *)&fifs->state[f2fs_ifs_iomap_longs(folio) + 1];
}
```

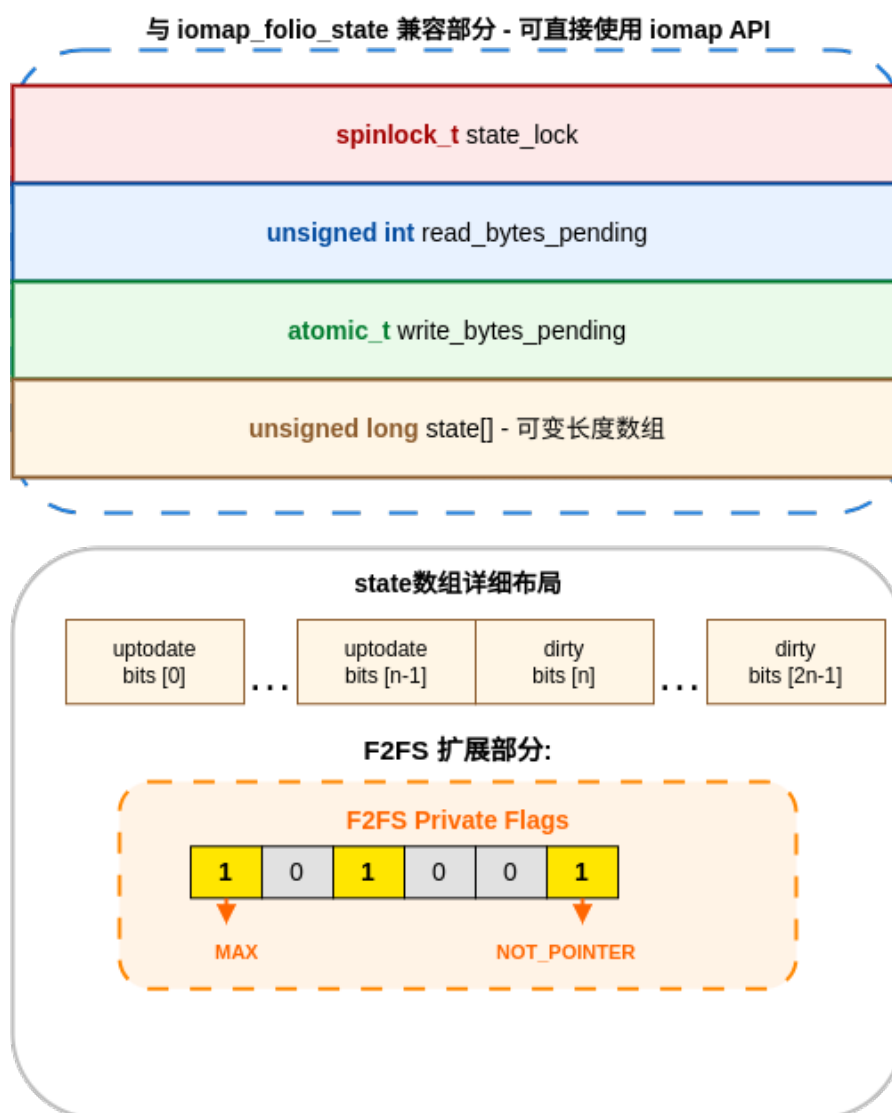


图 4-1 f2fs_folio_state 状态布局示意图

该结构可以按“固定头部 + 可变状态区”理解。固定头部保存并发控制和 pending 计数，可变状态区根据 folio 覆盖的文件系统块数动态扩展。

- **state_lock**: 保护读完成计数和状态位更新，主要用于 I/O 完成路径与文件系统路径并发访问的场景。
- **read_bytes_pending**: 记录当前 folio 中尚未完成读取的字节数。普通读和压缩读都会通过该字段避免提前调用 folio_end_read。
- **write_bytes_pending**: 记录写回提交中的字节数，用于高阶 folio 写回过程中的完成判断。
- **state[] 前半部分**: 保存 iomap 语义下的 uptodate / dirty 位图。代码中 f2fs_ifs_iomap_longs 按 $2 * \text{nr_blocks}$ 计算该区域需要的 unsigned long 数量。

state[] 后半部分：保存 F2FS 私有标志和 dirty_bytes_pending，其中私有标志用于 GC、atomic、inline 等路径。dirty_bytes_pending 用于压缩写回中的延迟解锁。

上述布局可以用伪代码概括为：

```
算法 4-1 f2fs_folio_state 的状态布局
输入: folio
nr_blocks <- i_blocks_per_folio(inode, folio)
iomap_bits <- 2 * nr_blocks
iomap_long <- BITS_TO_LONGS(iomap_bits)

header.state_lock <- protect pending counters and bitmaps
header.read_bytes_pending <- unfinished read bytes
header.write_bytes_pending <- unfinished writeback bytes

state[0, nr_blocks) <- uptodate bitmap
state[nr_blocks, 2 * nr_blocks) <- dirty bitmap
state[iomap_long] <- F2FS private flags
state[iomap_long + 1] <- dirty_bytes_pending
```

该布局把 iomap 位图放在前部，保证 ifs_set_range_uptodate、iomap_set_range_dirty 等接口仍可按预期访问。F2FS 额外状态放在后部，GC、压缩写回和原子写路径仍可通过统一接口读取私有标志。这样，folio->private 不再需要在“iomap 状态指针”和“F2FS 直接标志位”之间反复切换。

4.3.2 状态分配与旧私有状态迁移

状态分配由 f2fs_ifs_alloc 完成。这个函数的作用不只是分配内存，而是判断 folio->private 当前保存的状态类型。Large Folio 路径中，folio->private 可能为空，可能已经指向 F2FS 扩展状态，也可能指向 iomap 已经建立的状态对象。order-0 folio 还可能直接保存 F2FS 私有标志，因此该函数需要先完成分类，再决定复用、迁移或重新分配状态结构。

```
/* Source: fs/f2fs/f2fs_ifs.c:10-93, f2fs_ifs_alloc */
struct f2fs_folio_state *
f2fs_ifs_alloc(struct folio *folio, gfp_t gfp, bool force_alloc)
{
    struct f2fs_folio_state *fifs;
    void *old_private = folio_get_private(folio);
    size_t total_long;

    if (old_private) {
        unsigned int rbp =
            READ_ONCE(((struct f2fs_folio_state *)old_private)->
                read_bytes_pending);

        if (rbp == F2FS_IFS_MAGIC || rbp > F2FS_IFS_MAGIC)
```

```

        return old_private;

    total_longs = f2fs_ifs_total_longs(folio);
    fifs = kmalloc(sizeof(*fifs) +
        total_longs * sizeof(unsigned long), gfp);
    if (!fifs)
        return NULL;

    spin_lock_init(&fifs->state_lock);
    ifs_to_f2fs_ifs(old_private, fifs, folio);
    WRITE_ONCE(fifs->read_bytes_pending, F2FS_IFS_MAGIC);
    atomic_set(f2fs_ifs_dirty_bytes_pending_ptr(fifs, folio), 0);
    folio_change_private(folio, fifs);
    kfree(old_private);
    return fifs;
}

total_longs = f2fs_ifs_total_longs(folio);
fifs = kzalloc(sizeof(*fifs) + total_longs * sizeof(unsigned long),
    gfp);
if (!fifs)
    return NULL;

spin_lock_init(&fifs->state_lock);
if (folio_test_uptodate(folio))
    bitmap_set(fifs->state, 0, i_blocks_per_folio(folio_inode(folio),
        folio));
WRITE_ONCE(fifs->read_bytes_pending, F2FS_IFS_MAGIC);
atomic_set(&fifs->write_bytes_pending, 0);
atomic_set(f2fs_ifs_dirty_bytes_pending_ptr(fifs, folio), 0);
folio_attach_private(folio, fifs);
return fifs;
}

```

该函数的分支可以集中理解为以下四类：

已经是 F2FS 扩展状态：通过 `read_bytes_pending` 中的 `F2FS_IFS_MAGIC` 判断。若 `folio->private` 已经指向 `f2fs_folio_state`，直接返回原对象，避免重复分配。

已经存在 `iomap` 状态：分配新的 `f2fs_folio_state`，调用 `ifs_to_f2fs_ifs` 复制原有 `uptodate` / `dirty` 位图。然后用 `folio_change_private` 替换 `folio->private`。这个分支保证状态迁移时不丢失 `iomap` 已经维护的位图。

尚未存在私有状态：按 `folio` 覆盖的块数分配完整状态区，并根据 `folio_test_uptodate`、`folio_test_dirty` 初始化位图。再通过 `folio_attach_private` 绑定到 `folio`。

必须强制分配状态：当 order-0 folio 可能直接使用 folio->private 保存 F2FS 标志时，force_alloc 可以要求分配指针型状态对象。避免后续 iomap 路径把直接标志位误判为状态指针。

对应的伪代码如下：

算法 4-2 f2fs_ifs_alloc 的状态选择流程

输入：folio, gfp, force_alloc

输出：f2fs_folio_state 指针或 NULL

```
if folio is order-0 and force_alloc is false:
    return NULL

old_private <- folio_get_private(folio)

if old_private is f2fs_folio_state:
    return old_private

if old_private exists but is not f2fs_folio_state:
    fifs <- allocate f2fs_folio_state
    copy iomap bitmap from old_private to fifs
    initialize F2FS private flags and pending counters
    replace folio->private with fifs
    free old_private
    return fifs

fifs <- allocate empty f2fs_folio_state
initialize bitmap from folio uptodate and dirty flags
initialize F2FS private flags and pending counters
attach fifs to folio->private
return fifs
```

通过上述流程，f2fs_ifs_alloc 把 folio->private 的所有权转换集中到一个入口处理。普通读写路径、写回路径、GC 路径和压缩路径不需要分别判断 folio->private 当前属于谁。而是通过 f2fs_folio_state 访问统一的状态布局。

4.3.3 读完成计数

Large Folio 中一次读请求可能只覆盖 folio 的部分范围，也可能由多个 bio 分段完成。若仍然在任意一个 bio 完成时直接调用 folio_end_read，就可能提前释放 folio 锁或提前把 folio 标记为 uptodate。为此，读完成路径使用 read_bytes_pending 记录尚未完成的字节数。

```
/* Source: fs/f2fs/data.c:172-206, f2fs_iomap_finish_folio_read */
void f2fs_iomap_finish_folio_read(struct folio *folio,
                                  size_t off, size_t len, int error)
{
    struct f2fs_folio_state *fifs = folio->private;
    bool uptodate = !error;
```



```

bool finished = true;

if (folio_order(folio) > 0 && fifs) {
    unsigned long flags;

    spin_lock_irqsave(&fifs->state_lock, flags);
    fifs->read_bytes_pending -= len;
    if (!error)
        uptodate = ifs_set_range_uptodate(
            folio, (struct iomap_folio_state *)fifs,
            off, len);
    finished = fifs->read_bytes_pending == 0 ||
        fifs->read_bytes_pending == F2FS_IFS_MAGIC;
    spin_unlock_irqrestore(&fifs->state_lock, flags);

    if (finished)
        folio_end_read(folio, uptodate);
    return;
}

folio_end_read(folio, !error);
}

```

该函数根据 bio 完成时传入的 off 和 len 更新 folio 内的 uptodate 范围，并在所有相关字节完成后调用 folio_end_read。这样，完成路径的判断对象从“某个 page 是否完成”转为“folio 中本次读取覆盖的字节范围是否完成”。压缩读和普通读都可以复用这一完成语义。

4.4 脏页写回路径中的粒度控制

在 buffered write 完成后，用户数据首先停留在页缓存中，随后由写回路径写入磁盘。Large Folio 引入后，一个 folio 内可能只有部分文件系统块被修改。如果写回阶段直接把整个 folio 当作脏数据提交，就会放大 I/O 范围，也会影响压缩文件和 GC 路径对 folio 生命周期的判断。

本节的核心是 folio 内脏区定位和子范围写回。F2FS 在 f2fs_write_cache_folios 中使用 writeback_iter 遍历待写回 folio。再通过 f2fs_iomap_find_dirty_range 查找 folio 内的脏区间。普通文件脏区交给 f2fs_write_single_data_folio 提交；压缩文件脏区则先进入压缩上下文。由后续压缩写回路径统一处理。

4.4.1 从逐页写回到 Folio 内脏区定位

原有 f2fs_write_cache_pages 来自通用 write_cache_pages 的改造版本。该路径先通过 filemap_get_folios_tag 找到带 dirty 标记的 folio，再把每个 folio 展开为 page 数组。这种处理在 page 粒度路径中可以工作，但在 Large Folio 中会把一个 folio 内的多个子页重新拆散。

```

/* Source: fs/f2fs/data.c:3666-3791, f2fs_write_cache_pages */
nr_folios = filemap_get_folios_tag(mapping, &index, end,
                                   tag, &fbatch);
for (i = 0; i < nr_folios; i++) {
    struct folio *folio = fbatch.folios[i];

    idx = 0;
    p = folio_nr_pages(folio);
add_more:
    pages[nr_pages] = folio_page(folio, idx);
    folio_get(folio);
    if (++nr_pages == max_pages)
        goto write;
    if (++idx < p)
        goto add_more;
}

write:
for (i = 0; i < nr_pages; i++) {
    struct page *page = pages[i];
    struct folio *folio = page_folio(page);

    folio_lock(folio);
    if (!folio_clear_dirty_for_io(folio))
        continue;

    ret = f2fs_write_single_data_page(folio, &submitted,
                                      &bio, &last_block,
                                      wbc, io_type, 0, true);
}

```

这种 subpage 拆分会带来几类直接后果：

脏区信息被粗化。旧路径通过 `folio_clear_dirty_for_io` 清除 `folio` 的 `dirty` 状态。而不是在 `folio` 内保存 `dirty` bitmap。若一个高阶 `folio` 只有部分块被修改，旧路径无法把“哪些块需要写回”继续传递到提交阶段。

子页身份在提交阶段丢失。外层虽然把 `folio` 拆成 `pages[]`，但 `f2fs_write_single_data_page` 的接口仍接收 `folio`。函数内部又通过 `folio_page(folio, 0)` 取页。这样，非首页 subpage 的位置信息没有成为最终 I/O 提交参数。

同一 `folio` 会被重复处理。一个高阶 `folio` 展开后会在 `pages[]` 中出现多次。循环中会反复对同一个 `folio` 加锁、检查 `dirty/writeback` 状态。第一次清除 `dirty` 后，后续子页迭代可能只能走跳过或重试路径。生命周期判断与真实脏区不再一一对应。

压缩文件的簇级聚合被打散。压缩路径需要按 cluster 收集待写页。而 Large Folio 可能横跨多个 cluster。旧的 pages[] 视角只能逐页判断 cluster ready。难以表达同一 folio 内不同范围分别属于不同压缩簇的情况。

写回计数与提交范围不一致。旧路径按 submitted 调整 wbc->nr_to_write。而 Large Folio 下实际提交范围应由 folio 内 dirty range 决定。若仍按拆出的 page 计数，写回控制与真实 I/O 范围容易脱节。

新路径 f2fs_write_cache_folios 改为直接使用 writeback_iter。该迭代器返回的对象是 folio，写回逻辑也不再先拆成 page 数组，而是在 folio 内部继续查找 dirty range。写回入口的主体逻辑如下：

```
/* Source: fs/f2fs/data.c:3874-4010, f2fs_write_cache_folios */
while ((folio = writeback_iter(mapping, wbc, folio, &err))) {
    struct f2fs_folio_state *fifs = folio->private;
    u64 pos = folio_pos(folio);
    u64 end_pos = pos + folio_size(folio);
    u64 end_aligned;
    u32 r_len;

    if (!iomap_writepage_handle_eof(folio, inode, &end_pos)) {
        folio_unlock(folio);
        return 0;
    }

    if (i_blocks_per_folio(inode, folio) > 1 && !fifs) {
        fifs = f2fs_ifs_alloc(folio, 0, true);
        iomap_set_range_dirty(folio, 0, end_pos - pos);
    }

    folio_start_writeback(folio);
    end_aligned = round_up(end_pos, i_blocksize(inode));

    while ((r_len = f2fs_iomap_find_dirty_range(folio, &pos,
                                                end_aligned))) {
        err = f2fs_write_single_data_folio(folio, &submitted,
                                            NULL, NULL, wbc, io_type,
                                            0, false, false,
                                            pos, pos + r_len);

        if (err)
            goto handle_current_folio_error;
        pos += r_len;
    }

    iomap_clear_range_dirty(folio, 0, folio_size(folio));
    if (folio_test_locked(folio))
        folio_unlock(folio);
}
```

writeback_iter 负责从 address space 中取出需要写回的 folio。对于包含多个块的 folio，如果尚未分配 f2fs_folio_state，代码会先分配状态结构，并把当前有效范围标记为 dirty。随后，f2fs_iomap_find_dirty_range 在 folio 内寻找下一段脏区间，并将该范围传给 f2fs_write_single_data_folio。

这两条路径的差异集中在三个方面：

两条路径的遍历对象不同。旧路径虽然从 页缓存 中取出 folio，但随后会展开为 struct page *pages[]。新路径则保留 folio 作为 写回的基本对象。

两条路径的脏区表达也不同。旧路径依赖 folio_clear_dirty_for_io 处理整个 folio 的 dirty 状态。新路径通过 f2fs_iomap_find_dirty_range 在 folio 内部寻找实际脏区间。

提交入口不同：旧路径调用 f2fs_write_single_data_page。且没有向该函数传递 folio 内子范围；新路径调用 f2fs_write_single_data_folio，并显式传入 pos 与 pos + r_len。使提交范围能够落到 folio 内的子区间。

生命周期控制也不同。旧路径的锁、dirty 清理、写回结束和重试逻辑分散在逐页循环中。新路径把 folio 遍历、dirty range 查找和子范围提交放在同一个 folio 上下文内，因此更便于处理未提交范围的解锁和写回收尾。

这种处理方式把 写回的最小决策单位从整个 folio 收窄到 folio 内部的脏范围。对于普通文件而言，这使写回路径可以只围绕已修改的子范围构造写请求；对于后续压缩文件而言，它也提供了把同一 folio 中不同子范围加入压缩上下文的基础。

4.4.2 子范围提交与重试边界

f2fs_write_single_data_folio 接收 start 和 end 两个字节位置，并据此计算 folio 内需要提交的子页范围。函数内部先根据 folio_order(folio) 分流。order-0 folio 继续走原有单页写回函数，并保留单页路径中的解锁、等待、重新加锁和再检查流程。高阶 folio 则进入 folio 子范围提交路径。该分支会根据 start/end 限定写回边界，并在提交失败或部分提交时回到统一的重试处理。

```
/* Source: fs/f2fs/data.c:3410-3522, f2fs_write_single_data_folio */
int f2fs_write_single_data_folio(struct folio *folio, int *submitted_pages_count,
                                struct bio **bio_ptr, sector_t *last_block_ptr,
                                struct writeback_control *wbc,
                                enum iostat_type io_type, int compr_blocks,
                                bool from_compress, bool is_reclaim,
                                u64 start, u64 end)
{
    struct inode *inode = folio->mapping->host;
    struct f2fs_sb_info *sbi = F2FS_I_SB(inode);
```

```

int err = 0;
int local_submitted_count = 0;
if (folio_order(folio) == 0) {
#ifdef CONFIG_F2FS_DEBUG_PRINT
f2fs_err(sbi, "f2fs_write_single_data_folio called with folio_order 0");
#endif
retry_single:
    err = f2fs_write_single_data_page(folio, &local_submitted_count,
                                      bio_ptr, last_block_ptr,
                                      wbc, io_type,
                                      compr_blocks,
                                      !is_reclaim);

    if (err == -EAGAIN) {
        // f2fs_write_single_data_page does not unlock on -EAGAIN.
        // We must unlock it before waiting.
        folio_unlock(folio);
        /* may come from f2fs_write_data_pages */
        if (wbc->sync_mode == WB_SYNC_ALL || f2fs_compressed_file(inode)) {
            f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT);
            folio_lock(folio);
            if (unlikely(folio->mapping != inode->i_mapping ||
                        !folio_test_dirty(folio))) {
                return 0;
            }
            goto retry_single;
        }
        return -EAGAIN;
    }
    if (submitted_pages_count)
        *submitted_pages_count = local_submitted_count;
    return err;
}
#ifdef CONFIG_F2FS_DEBUG_PRINT
f2fs_err(sbi, "f2fs_write_single_data_folio called with folio_order:%d",
folio_order(folio));
#endif
struct f2fs_folio_state *fifs = folio->private;
WARN_ON_ONCE(!fifs);
pgoff_t folio_start_idx = (start - folio_pos(folio)) >> PAGE_SHIFT;
pgoff_t folio_end_idx = (end - 1 - folio_pos(folio)) >> PAGE_SHIFT;

for (pgoff_t i = folio_start_idx; i <= folio_end_idx; ++i) {
    struct page *page = folio_page(folio, i);
    struct f2fs_io_info fio = {
        .sbi = sbi,
        .ino = inode->i_ino,
        .type = DATA,
        .op = REQ_OP_WRITE,
    };

```

```

        .op_flags = wbc_to_write_flags(wbc),
        .old_blkaddr = NULL_ADDR,
        .page = page,
        .encrypted_page = NULL,
        .submitted = 0,
        .compr_blocks = compr_blocks,
        .need_lock = compr_blocks ? LOCK_DONE : LOCK_RETRY,
        .meta_gc = f2fs_meta_inode_gc_required(inode) ? 1 : 0,
        .io_type = io_type,
        .io_wbc = wbc,
        .bio = bio_ptr ? bio_ptr : NULL,
        .last_block = last_block_ptr
    };

retry_multi:
    atomic_add(PAGE_SIZE, &fifs->write_bytes_pending);
#ifdef CONFIG_F2FS_DEBUG_PRINT
    FUNC(print_wbp, folio);
#endif
    err=f2fs_do_write_data_page(&fio);

/* No more unlock for large folio in retry write logic for now.
Because it will cause iomap_folio_state to become modified by buffered write
and handle partial truncate is tricky. There's no much benefit to unlock a large folio
here*/

    if (err == -EAGAIN) {
        if (wbc->sync_mode == WB_SYNC_ALL || from_compress) {
            f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT);
            fio.need_lock = LOCK_REQ;
            goto retry_multi;
        } else {
            err = -EAGAIN;
            goto handle_multi_err;
        }
    }

    //TODO : for inplace write we should update bio and last_block. Need further
research.

    // if (bio_ptr) *bio_ptr = fio.bio;
    // if (last_block_ptr) *last_block_ptr = fio.last_block;
handle_multi_err:
    if (err) {
        if (fifs) {
            // If submission failed for this page, it won't go to
f2fs_write_end_io

            // for this portion.
            if (!atomic_sub_and_test(PAGE_SIZE, &fifs-
>write_bytes_pending)) {
                folio_end_writeback(folio);
            }

```

```

        bio_endio(*fio.bio);
    }
    break;
} else {
    local_submitted_count++;
}
}

if (submitted_pages_count)
    *submitted_pages_count = local_submitted_count;

return err;
}

```

这段代码的第一层分流由 `folio_order(folio)` 决定。order-0 folio 仍调用 `f2fs_write_single_data_page`。这样普通 4 KiB 页的写回路径不会被强行改造成高阶 folio 逻辑。该分支遇到 `-EAGAIN` 时会先解锁 folio。再在同步写回或压缩文件场景下等待并重新加锁；重新加锁后还要检查 `folio->mapping` 是否仍指向当前 inode，以及 folio 是否仍为 `dirty`。这个检查用于处理等待期间 folio 被截断、迁移或被其他路径清理的情况。

高阶 folio 分支的处理方式不同。函数先通过 `start`、`end` 和 `folio_pos(folio)` 计算 `folio_start_idx` 与 `folio_end_idx`，只遍历当前脏区间覆盖到的子页。每个子页都会构造一个 `f2fs_io_info`，但这些提交仍共享同一个高阶 folio 的状态对象。提交前增加 `write_bytes_pending`，是为了让后续写回完成路径知道该 folio 内还有多少字节处于 I/O 提交或等待完成状态。

高阶 folio 的 `retry` 策略没有沿用 order-0 分支中的“解锁后等待”方式。代码注释中说明，在高阶 folio 的 `retry` 过程中暂不解锁。因为解锁后 `buffered write` 可能再次修改 folio 私有状态，部分截断场景也会变得难以处理，因此，高阶 folio 遇到 `-EAGAIN` 时。只在同步写回或压缩写回场景下等待后重试，并把 `fio.need_lock` 调整为 `LOCK_REQ`；其他场景则进入错误处理分支。

错误处理也围绕高阶 folio 的 `pending` 计数展开。若某个子页提交失败，它不会进入正常的 `f2fs_write_end_io` 完成路径，因此函数需要主动减少 `write_bytes_pending`。当减少后仍有未完成字节时，代码调用 `folio_end` 写回结束当前 folio 的写回状态，并通过 `bio_endio` 结束对应 bio。这样，子页提交失败不会让整个高阶 folio 长时间停留在 `writeback` 状态。

4.4.3 压缩上下文中的延迟解锁

普通文件的写回可以在子范围提交后释放 folio 锁，而压缩文件写回需要先把若干 folio 子范围聚合到压缩上下文中。此时 folio 已经被 `writeback` 选中，但真正的

I/O 提交要等压缩簇组装完成后才能发生。为避免提前解锁，代码使用 `dirty_bytes_pending` 和 `deferred_unlock` 记录该 folio 仍有脏字节处于压缩上下文中。

```
/* Source: fs/f2fs/data.c:3970-4001, f2fs_write_cache_folios */
add_len = f2fs_compress_ctx_add_folio(&cc, folio, pos, r_len);
if (folio_order(folio) > 0 && fifs)
    atomic_add(add_len,
               f2fs_ifs_dirty_bytes_pending_ptr(fifs, folio));

if (!err && fifs && folio_order(folio) > 0 &&
    atomic_read(f2fs_ifs_dirty_bytes_pending_ptr(fifs, folio)) > 0)
    f2fs_set_folio_private_deferred_unlock(folio);
```

压缩簇真正写出时，`f2fs_write_raw_pages` 会按 folio 统计属于同一 folio 的页数。调用 `f2fs_write_single_data_folio` 提交对应范围，并在提交完成后减少 `dirty_bytes_pending`。当计数归零且 folio 带有 `deferred_unlock` 标志时，代码清除标志并解锁 folio。

```
/* Source: fs/f2fs/compress.c:1728-1766, f2fs_write_raw_pages */
ret = f2fs_write_single_data_folio(folio, &submitted, NULL,
                                   NULL, wbc, io_type, compr_blocks,
                                   true, false, start, end);

if (folio_order(folio) > 0 && fifs) {
    if (atomic_sub_and_test(submitted << PAGE_SHIFT,
                           f2fs_ifs_dirty_bytes_pending_ptr(fifs, folio)) &&
        f2fs_folio_private_deferred_unlock(folio)) {
        f2fs_clear_folio_private_deferred_unlock(folio);
        folio_unlock(folio);
    }
}
```

这一机制把 folio 生命周期和压缩簇提交过程关联起来。写回入口可以继续按 folio 遍历脏页，而压缩路径可以在簇级别完成实际提交，二者通过 `dirty_bytes_pending` 保持状态一致。

4.5 GC 路径中的 Large Folio 适配

F2FS 的 GC 路径负责搬迁有效数据块。传统实现中，GC 以单个 block 或 page 为主要对象，读出旧块、写入新块，并更新元数据。Large Folio 引入后，页缓存中的一个 folio 可能覆盖多个块，GC 如果仍然把 `folio->page` 当作唯一对象处理。就需要明确本次搬迁对应 folio 内的哪个子页范围。

本节将 GC 适配分为两部分。第一，GC 标志需要纳入 `f2fs_folio_state` 的私有标志区域，避免与 `iomap` 状态冲突。第二，GC 的脏标记和 bio 提交需要携带 folio 内偏移，避免把整个 folio 错当成单个 page 处理。

4.5.1 GC 搬运中的私有状态

后台 GC 中，`move_data_page` 在拿到目标 folio 后，会为该 folio 设置 `gcing` 标志。对于 `order-0` folio，代码沿用 `folio_mark_dirty`；对于高阶 folio，代码转为标记 folio 内的子范围。

```
/* Source: fs/f2fs/gc.c:1457-1530, move_data_page */
if (gc_type == BG_GC) {
    if (folio_test_writeback(folio)) {
        err = -EAGAIN;
        goto out;
    }

    f2fs_set_folio_private_gcing(folio);
    if (folio_order(folio) == 0)
        folio_mark_dirty(folio);
    else
        iomap_set_range_dirty(folio, bidx << PAGE_SHIFT,
                               PAGE_SIZE);
}
```

这里的 `f2fs_set_folio_private_gcing` 最终使用 `f2fs_folio_state` 中的私有标志区域。这样。GC 可以继续使用 F2FS 原有的迁移状态判断，同时不覆盖 `iomap` 的 `uptodate/dirty` 位图。对于高阶 folio，`iomap_set_range_dirty` 表示本次 GC 只影响一个子页范围，后续 `writeback` 可以根据 `dirty bitmap` 定位需要写出的区间。

4.5.2 GC 读路径与 bio 提交偏移

GC 预读路径 `ra_data_block` 仍以单个逻辑块为入口。它获取 页缓存 中的 folio，查找该逻辑块对应的数据块地址，并通过 `f2fs_submit_page_bio` 提交读取。

```
/* Source: fs/f2fs/gc.c:1208-1290, ra_data_block */
folio = f2fs_grab_cache_folio(mapping, index, true);
if (IS_ERR(folio))
    return PTR_ERR(folio);

fio.page = &folio->page;
fio.new_blkaddr = fio.old_blkaddr = dn.data_blkaddr;

fio.encrypted_page = f2fs_pagecache_get_page(META_MAPPING(sbi),
                                              dn.data_blkaddr,
                                              FGP_LOCK | FGP_CREAT,
                                              GFP_NOFS);
if (!fio.encrypted_page)
    return -ENOMEM;

err = f2fs_submit_page_bio(&fio);
```

GC 读写路径仍然调用 `f2fs_submit_page_bio`，但该函数在向 bio 中加入数据时使用 folio 内页索引计算偏移。

```

/* Source: fs/f2fs/data.c:753-785, f2fs_submit_page_bio */
int f2fs_submit_page_bio(struct f2fs_io_info *fio)
{
    struct folio *fio_folio = page_folio(fio->page);
    struct folio *data_folio = fio->encrypted_page ?
        page_folio(fio->encrypted_page) : fio_folio;
    struct page *page = fio->encrypted_page ?
        fio->encrypted_page : fio->page;
    struct bio *bio;

    bio = __bio_alloc(fio, 1);
    bio_add_folio_nofail(bio, data_folio, PAGE_SIZE,
        folio_page_idx(data_folio, page) << PAGE_SHIFT);

    if (is_read_io(bio_op(bio)))
        f2fs_submit_read_bio(fio->sbi, bio, fio->type);
    else
        f2fs_submit_write_bio(fio->sbi, bio, fio->type);

    fio->submitted = true;
    return 0;
}

```

该偏移计算保证 bio 中加入的是目标 folio 内的一个 page 大小范围，而不是默认从 folio 起始位置提交。GC 搬迁的对象是具体逻辑块；若 bio 偏移与实际子页不一致，读取或写出的数据就可能落到 folio 的错误位置。

4.5.3 前台搬运中的写回控制

前台 GC 路径需要直接写出目标数据页。代码在写出前等待已有 writeback，随后标记 folio dirty。清除 dirty-for-io 状态并设置 gcing 标志，再调用 f2fs_do_write_data_page 完成写出。

```

/* Source: fs/f2fs/gc.c:1490-1526, move_data_page */
retry:
    f2fs_folio_wait_writeback(folio, DATA, true, true);

    folio_mark_dirty(folio);
    if (folio_clear_dirty_for_io(folio)) {
        inode_dec_dirty_pages(inode);
        f2fs_remove_dirty_inode(inode);
    }

    f2fs_set_folio_private_gcing(folio);

    err = f2fs_do_write_data_page(&fio);
    if (err) {
        f2fs_clear_folio_private_gcing(folio);
        if (err == -ENOMEM) {

```

```

        memalloc_retry_wait(GFP_NOFS);
        goto retry;
    }
    if (is_dirty)
        folio_mark_dirty(folio);
}

```

这段逻辑保留了 F2FS GC 对 writeback 和元数据一致性的控制，同时把 gcing 状态纳入统一的 folio 私有状态管理。后台 GC 侧重标记子范围并交由后续 writeback 处理，前台 GC 则在等待和重试控制后直接提交写出。两种路径的共同点是都不能把 Large Folio 简化成单个 page，而必须明确本次操作涉及的 folio 内范围和状态。

4.6 压缩文件场景下的扩展处理

压缩文件在普通 iomap 路径之外多了一层簇级语义：F2FS 的压缩数据以 cluster 为单位组织。而 Large Folio 以页缓存中的 folio 为单位组织。一个 folio 可能覆盖多个压缩簇，一个压缩簇也可能只占 folio 的一部分，因此，压缩文件需要增加簇级读取、部分簇补齐和延迟解锁处理。

4.6.1 压缩读取中的簇级映射与完成计数

压缩读取路径需要先判断当前范围属于普通映射还是压缩簇映射。普通文件读取只需要返回逻辑块到物理块的区间映射。压缩文件读取还要识别 cluster head、压缩物理块数量，以及当前 Large Folio 能覆盖的压缩簇个数。因此，f2fs_buffered_read_compress_iomap_begin 在 iomap 迭代开始阶段先完成簇级判断，再把结果保存到读取上下文中。

```

/* Source: fs/f2fs/data.c:5183-5323,
f2fs_buffered_read_compress_iomap_begin */
static int f2fs_buffered_read_compress_iomap_begin(struct inode *inode,
                                                    loff_t pos, loff_t length,
                                                    unsigned flags,
                                                    struct iomap *iomap,
                                                    struct iomap *srcmap)
{
    struct f2fs_sb_info *sbi = F2FS_I_SB(inode);
    struct iomap_iter *iter = container_of(iomap,
                                            struct iomap_iter, iomap);
    struct f2fs_readpage_ctx *ctx =
        (struct f2fs_readpage_ctx *)iter->private;
    pgoff_t index = pos >> PAGE_SHIFT;
    loff_t map_len_bytes;
    int log_cluster_size = ctx->cc.log_cluster_size;
    int cluster_size = ctx->cc.cluster_size;
    int ret = 0;

    iomap->bdev = sbi->sb->s_bdev;

```

```

iomap->private = NULL;
iomap->flags = 0;
iomap->offset = F2FS_BLK_TO_BYTES(F2FS_BYTES_TO_BLK(pos));

if (f2fs_compressed_file(inode)) {
    pgoff_t cluster_start_idx;
    int all_clusters;

    if (ctx->cur_folio &&
        offset_in_folio(ctx->cur_folio, iter->pos) == 0)
        ctx->cur_folio = NULL;
    if (!ctx->cur_folio) {
        ctx->cur_folio = readahead_folio(ctx->rac);
        ctx->cur_folio_in_bio = false;
        ctx->cur_folio_in_compress_ctx = true;
    }

    cluster_start_idx = cluster_idx(&ctx->cc, index);
    all_clusters =
        (folio_nr_pages(ctx->cur_folio) -
         (index - ctx->cur_folio->index)) >>
        log_cluster_size;

    if (index != (cluster_start_idx << log_cluster_size) ||
        all_clusters <= 1) {
        if (!ctx->dn.node_folio) {
            ret = f2fs_get_dnode_of_data(
                &ctx->dn,
                cluster_start_idx << log_cluster_size,
                LOOKUP_NODE_RA);
            if (ret)
                return ret;
            ctx->has_put_dn = false;
        }

        if (data_blkaddr(inode, ctx->dn.node_folio,
            ctx->dn.ofs_in_node) ==
            COMPRESS_ADDR) {
            ctx->compressed_clusters = 1;
            map_len_bytes = min_t(
                loff_t, length,
                (((loff_t)(cluster_start_idx + 1)
                 << log_cluster_size) -
                 index)
                 << PAGE_SHIFT);
            iomap->length = map_len_bytes;
            iomap->type = IOMAP_MAPPED;
            return 0;
        }
    }
}

```

```

    }
} else {
    struct f2fs_compressed_pblks_info *pblks_info =
        ctx->pblks_arr;
    struct folio *nfolio = ctx->dn.node_folio;
    unsigned int ofs_in_node = ctx->dn.ofs_in_node;
    unsigned int end_ofs = ctx->dn.node_folio ?
        ADDRS_PER_PAGE(&ctx->dn.node_folio->page,
            inode) :
        0;
    int process_clusters = 0;

    all_clusters = min_t(int, all_clusters,
        MAX_PBLKS_INFO_SIZE);
    for (int i = 0; i < all_clusters; i++) {
        int cluster_idx = cluster_start_idx + i;
        struct extent_info ei = {};
        unsigned int max_possible_pblks;

        pblks_info->num_pblks = 0;
        if (f2fs_lookup_read_extent_cache(
            inode, cluster_idx, &ei)) {
            pblks_info->is_from_extent = true;
            pblks_info->blk = ei.blk;
            pblks_info->num_pblks = ei.c_len;
        } else {
            if (!nfolio) {
                ret = f2fs_get_dnode_of_data(
                    &ctx->dn,
                    cluster_idx <<
                        log_cluster_size,
                    LOOKUP_NODE_RA);
                if (ret)
                    return ret;
                ctx->has_put_dn = false;
                ofs_in_node = ctx->dn.ofs_in_node;
                end_ofs = ADDRS_PER_PAGE(
                    &ctx->dn.node_folio->page,
                    inode);
                nfolio = ctx->dn.node_folio;
            }
            pblks_info->is_from_extent = false;
            if (data_blkaddr(inode, nfolio,
                ofs_in_node) !=
                COMPRESS_ADDR)
                break;
        }
    }
}

```

```

        max_possible_pblks =
            pblks_info->is_from_extent ?
                ei.c_len :
                cluster_size;
        for (int j = 0; j < max_possible_pblks; j++) {
            block_t blkaddr;

            if (pblks_info->is_from_extent)
                blkaddr = ei.blk + j;
            else
                blkaddr = data_blkaddr(
                    inode, nfolio,
                    ofs_in_node + j + 1);
            if (!__is_valid_data_blkaddr(blkaddr))
                break;
            if (!f2fs_is_valid_blkaddr(
                sbi, blkaddr,
                DATA_GENERIC)) {
                if (!pblks_info->is_from_extent) {
                    f2fs_put_dnode(&ctx->dn);
                    ctx->has_put_dn = true;
                }
                return -EFAULT;
            }
            pblks_info->num_pblks++;
            pblks_info->dn = &ctx->dn;
        }
        process_clusters++;
        pblks_info++;
        ofs_in_node += cluster_size;
        if (ofs_in_node >= end_ofs)
            break;
    }
    ctx->compressed_clusters = process_clusters;
    iomap->length =
        process_clusters << log_cluster_size << PAGE_SHIFT;
    iomap->type = IOMAP_MAPPED;
    return 0;
}

return f2fs_buffered_read_iomap_ops.iomap_begin(
    inode, pos, length, flags, iomap, srcmap);
}

```

这段逻辑把压缩读取的入口分成三类。第一类是单簇压缩读取：当当前位置不在簇起点，或当前 folio 只覆盖一个压缩簇时，函数通过 ctx->dn 查 cluster head。若该位置为 COMPRESS_ADDR，则设置 ctx->compressed_clusters = 1，并把 iomap-

>length 限定在当前压缩簇边界内。第二类是多簇压缩读取：当 Large Folio 覆盖多个压缩簇时，函数把多个簇的物理块信息保存到读取上下文中。

多簇分支是压缩读取优化的关键。该分支没有把压缩数据伪装成一个普通连续磁盘地址，而是把压缩簇数量和物理块列表保存在 f2fs_readpage_ctx 中。后续读取迭代器可以在一次 Large Folio 读取范围内连续处理多个压缩簇，从而减少按单簇反复查找 dnode 或 extent cache 的次数。

完成簇级映射后，f2fs_compress_readpage_iter 根据 ctx->compressed_clusters 选择读取方式。如果该值非零，说明前一阶段已经识别出压缩簇，读取路径进入 f2fs_do_read_multi_folios；否则说明当前区间按普通映射读取。路径进入 f2fs_do_read_single_folio_iomap。

```
/* Source: fs/f2fs/data.c:5328-5360, f2fs_compress_readpage_iter */
static int f2fs_compress_readpage_iter(struct iomap_iter *iter,
                                       struct f2fs_readpage_ctx *ctx)
{
    loff_t pos = iter->pos;
    loff_t length = iomap_length(iter);
    struct folio *folio = ctx->cur_folio;
    struct f2fs_folio_state *fifs;
    size_t poff, plen;
    int ret;

    fifs = f2fs_ifs_alloc(folio, iter->flags, false);
    iomap_adjust_read_range(iter->inode, folio, &pos,
                           length, &poff, &plen);

    if (plen == 0)
        goto done;

    if (ctx->compressed_clusters)
        ret = f2fs_do_read_multi_folios(ctx, pos, plen);
    else
        ret = f2fs_do_read_single_folio_iomap(iter, ctx,
                                              pos, plen, poff);

    if (ret)
        return ret;

done:
    length = pos - iter->pos + plen;
    return iomap_iter_advance(iter, &length);
}
```

在多簇读取中，代码会把 folio 加入压缩上下文，并按照 ctx->compressed_clusters 增加 read_bytes_pending。后续每个压缩簇仍按自身的压缩物理块列表构造 bio 和解压上下文，但 folio 完成判断由同一个 pending 计数统一约束。

```

/* Source: fs/f2fs/data.c:5427-5493, f2fs_do_read_multi_folios */
if (ctx->compressed_clusters == 1) {
    loff_t add_len = f2fs_compress_ctx_add_folio(cc, folio, pos, plen);

    if (folio_order(folio) > 0 && fifs) {
        spin_lock_irq(&fifs->state_lock);
        fifs->read_bytes_pending += add_len;
        spin_unlock_irq(&fifs->state_lock);
    }

    ret = f2fs_read_multi_folios(cc, &ctx->pblks_arr[0],
                                &ctx->dn, bio_ret, nr_pages,
                                rac, false);
} else {
    if (folio_order(folio) > 0 && fifs) {
        spin_lock_irq(&fifs->state_lock);
        fifs->read_bytes_pending +=
            ctx->compressed_clusters << PAGE_SHIFT <<
            cc->log_cluster_size;
        spin_unlock_irq(&fifs->state_lock);
    }

    for (int i = 0; i < ctx->compressed_clusters; i++) {
        f2fs_compress_ctx_add_folio(cc, folio, pos, plen);
        ret = f2fs_read_multi_folios(cc, &ctx->pblks_arr[i],
                                    &ctx->dn, bio_ret, nr_pages,
                                    rac, false);

        if (ret)
            return ret;
        cc->cluster_idx++;
        pos += cluster_size << PAGE_SHIFT;
        f2fs_reset_compress_ctx(cc, true);
        ctx->dn.ofs_in_node += cluster_size;
    }
}

```

普通映射读取则把本次读取长度加入 `read_bytes_pending`，随后将 `folio`、长度和 `folio` 内偏移加入 `bio`。`bio` 完成后，前文所述的 `f2fs_iomap_finish_folio_read` 会根据相同的范围减少 `pending` 计数。

```

/* Source: fs/f2fs/data.c:5539-5580, f2fs_do_read_single_folio_iomap */
if (fifs) {
    spin_lock_irq(&fifs->state_lock);
    fifs->read_bytes_pending += plen;
    spin_unlock_irq(&fifs->state_lock);
}

if (!bio_add_folio(ctx->bio, folio, plen, poff))
    goto submit_and_realloc;

```


通过上述分层，压缩读取中的映射判断和 I/O 提交被分开处理。f2fs_buffered_read_compress_iomap_begin 负责识别普通区间、单个压缩簇和多个压缩簇。f2fs_compress_readpage_iter 根据识别结果进入普通 bio 读取或压缩簇读取。read_bytes_pending 则保证 Large Folio 的完成时机不会早于内部子区间 I/O。

4.6.2 压缩写入中的 Large Folio 准备

压缩写入比普通写入多了 read-modify-write 过程。若写入只覆盖压缩簇的一部分，文件系统需要先读取原有簇内容，解压后在页缓存中补齐未写入部分，再让用户写入覆盖目标范围。Large Folio 场景下，这个过程还要考虑 folio 与 cluster 的边界不一致。

实现中，压缩写入分为头部部分簇、中间完整簇和尾部部分簇。部分簇需要读取原有压缩数据；完整簇可以直接标记对应范围为 uptodate；最后通过 read_bytes_pending 的偏置等待必要读取完成。

```
/* Source: fs/f2fs/data.c:5666-5874, f2fs_buffered_write_iomap_begin */
folio = iomap_get_folio(iter, offset, length);
if (IS_ERR(folio))
    return PTR_ERR(folio);

fifs = f2fs_ifs_alloc(folio, iter->flags, false);
if (folio_order(folio) > 0 && fifs) {
    spin_lock_irqsave(&fifs->state_lock, flags);
    fifs->read_bytes_pending += 1;
    spin_unlock_irqrestore(&fifs->state_lock, flags);
}

head_cluster_start = start_idx & ~((pgoff_t)cluster_size - 1);
tail_cluster_start =
    (folio->index + folio_nr_pages(folio) - 1) &
    ~((pgoff_t)cluster_size - 1);

head_partial = start_idx != head_cluster_start;
tail_partial = end_idx - tail_cluster_start + 1 < cluster_size;
```

部分簇处理时，代码把相关 folio 加入压缩上下文，并根据 folio 内子页的 uptodate 位判断是否需要真正读取压缩簇。中间完整簇不需要读取旧数据，可以直接设置 folio 内对应范围的 uptodate 位。

```
/* Source: fs/f2fs/data.c:5772-5813, f2fs_buffered_write_iomap_begin */
if (!all_uptodate) {
    struct f2fs_compressed_pblks_info pi = { .dn = &dn };

    cc.cluster_idx = (idx - 1) >> log_cluster_size;
    err = f2fs_read_multi_folios(&cc, &pi, &dn, &bio,
                                folio_nr_pages(folio), NULL, false);
    if (err)
        goto out_err_unlock;
```

```

}

iomap_set_range_uptodate(folio, coff,
                        (size_t)cluster_size << PAGE_SHIFT);
iomap->length += cluster_size << PAGE_SHIFT;

```

在读取提交后，代码等待 `read_bytes_pending` 回到带偏置的完成状态，再撤销偏置并把 `folio` 标记为 `uptodate`。这个偏置的作用是区分“没有发起额外读取”和“读取已经完成”两种情况，避免在异步 I/O 完成前提前进入用户写入阶段。

```

/* Source: fs/f2fs/data.c:5852-5874, f2fs_buffered_write_iomap_begin */
if (bio)
    f2fs_submit_read_bio(sbi, bio, DATA);

if (folio_order(folio) > 0 && fifs) {
    for (;;) {
        bool done;

        spin_lock_irqsave(&fifs->state_lock, flags);
        done = fifs->read_bytes_pending == F2FS_IFS_MAGIC + 1;
        spin_unlock_irqrestore(&fifs->state_lock, flags);
        if (done)
            break;
        f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT);
    }

    spin_lock_irqsave(&fifs->state_lock, flags);
    fifs->read_bytes_pending -= 1;
    folio_mark_uptodate(folio);
    spin_unlock_irqrestore(&fifs->state_lock, flags);
}

```

4.6.3 压缩写回中的簇级提交

压缩写回在 4.4 节已经涉及，这里从压缩簇角度补充说明。`f2fs_write_raw_pages` 会遍历压缩上下文中的 `rpages`。把属于同一 `folio` 的连续页合并为一个提交范围，然后调用 `f2fs_write_single_data_folio`。这使压缩路径可以复用普通写回中的子范围提交逻辑。

```

/* Source: fs/f2fs/compress.c:1718-1736, f2fs_write_raw_pages */
while ((i + num_to_skip) < cc->cluster_size &&
       cc->rpages[i + num_to_skip] &&
       page_folio(cc->rpages[i + num_to_skip]) == folio)
    num_to_skip++;

start = (folio->index +
        folio_page_idx(folio, cc->rpages[i])) << PAGE_SHIFT;
end = (folio->index +
       folio_page_idx(folio, cc->rpages[i]) +
       num_to_skip) << PAGE_SHIFT;

```

```
ret = f2fs_write_single_data_folio(folio, &submitted, NULL,
                                   NULL, wbc, io_type,
                                   compr_blocks, true, false,
                                   start, end);
```

提交成功后，压缩路径根据已经提交的页数减少 `dirty_bytes_pending`。当该 `folio` 在压缩上下文中的脏字节全部处理完毕后，再清除延迟解锁标志并释放 `folio`。这样，压缩文件的簇级处理不会破坏 `Large Folio` 的生命周期管理。

第五章 项目性能测试

本章说明本文使用的顺序读写测试方法，并给出 QEMU 虚拟机和树莓派 5 平台上的结果。测试数据来自 `fio` 日志，后续图表由脚本从日志中解析生成。

5.1 测试环境与方法论

5.1.1 顺序读写测试的批量调度

`run_benchmark.py` 负责展开本次实验的参数组合。脚本在测试前记录 `vm.dirty_*` 参数，测试结束后恢复原值；随后按内核版本、读写模式、队列深度和块大小调用 `fio_test.sh`。读测试分为 `COLD READ` 和 `WARM READ`：前者在每轮测试前清空缓存，后者保留已有页缓存。

脚本内部通过 `original_vm_settings` 记录测试前的 `vm.dirty_*` 参数。测试结束后，`restore_write_fence()` 会恢复原状，避免测试过程改变系统后续状态。`run_single_test()` 将抽象队列深度映射到 `fio` 的 `numjobs` 或 `iodepth` 参数上。因此，同一套测试入口可以兼容 `psync` 与 `io_uring` 等不同 `ioengine`。下面摘录测试调度脚本中的核心函数代码：

```
// File: tools/run_benchmark.py

def restore_write_fence():

    global original_vm_settings
    if not original_vm_settings:
        return

    print("\n--- [FENCE DOWN] Restoring original kernel parameters... ---")
    try:
        for param, value in original_vm_settings.items():
            print(f" Restoring {param} = {value}")
```

```

        subprocess.run(
            ["sudo", "sysctl", "-w", f"{param}={value}"],
            check=True,
            capture_output=True,
        )
    print("--- Kernel parameters restored successfully. ---")
    original_vm_settings = {}
except (subprocess.CalledProcessError, FileNotFoundError) as e:
    print(f"[!!!] CRITICAL ERROR: Failed to restore kernel parameters: {e}",
          file=sys.stderr)
    print("[!!!] Please restore them manually:", file=sys.stderr)
    for param, value in original_vm_settings.items():
        print(f" sudo sysctl -w {param}={value}", file=sys.stderr)
    sys.exit(1)

def get_system_memory_gb():
    mem_bytes = psutil.virtual_memory().total
    gb_rounded = math.ceil(mem_bytes / (1024**3))
    return f"{gb_rounded}G"

def run_single_test(kernel_ver, mem_config, rw_mode,
                    bs, file_path, ioengine, qd):

    if ioengine == "psync":
        numjobs = qd
        iodepth = 1
    else:
        numjobs = 1
        iodepth = qd

    print(
        f" -> Running: bs={bs}, mode={rw_mode}, kernel={kernel_ver}, "
        f"mem={mem_config}, file={file_path}, engine={ioengine}, qd={qd} "
        f"(iodepth={iodepth}, numjobs={numjobs})"
    )

    title = f"{kernel_ver}_{mem_config}_mem"
    extra_fio_flags = []
    if rw_mode == "r":

```

```

extra_fio_flags += [
    "--readonly=1",
    "--allow_file_create=0",
    "--unlink=0",
]

try:
    subprocess.run(
        [
            "sudo", "bash", FIO_SCRIPT_PATH,
            title, bs, rw_mode,
            f"file={file_path}",
            f"ioengine={ioengine}",
            f"iodepth={iodepth}",
            f"numjobs={numjobs}",
            f"qd={qd}",
            *extra_fio_flags,
        ],
        check=True,
        capture_output=False,
        text=True,
    )
except subprocess.CalledProcessError as e:
    print(f" [!] ERROR running test for bs={bs}, qd={qd}.")
    print(f" [!] STDERR: {e.stderr}")
    return False
return True

def main():
    parser = argparse.ArgumentParser(
        description="Automated FIO benchmark orchestrator."
    )
    parser.add_argument(
        "--name", "-n", required=True,
        help="Kernel version identifier (e.g. 'vanilla', 'iomap_v1').",
    )
    parser.add_argument(
        "--type", "-t", required=True,
        choices=["read", "write", "sync", "r", "w", "s"],
        help="Test type: read/write/sync.",
    )
    parser.add_argument(
        "--file", "-f",
        default=DEFAULT_FILE,

```

```

        help=f"Target file for FIO (default: {DEFAULT_FILE})",
    )
    parser.add_argument(
        "--ioengine", "-e",
        choices=["psync", "io_uring", "libaio"],
        default="psync",
        help="fio ioengine.",
    )
    parser.add_argument(
        "--qdsset",
        default="1",
        help="Comma-separated queue depths to sweep.",
    )
    parser.add_argument(
        "--qdwwrite",
        action="store_true",
        help="Also sweep queue depths for WRITE tests.",
    )

args = parser.parse_args()
try:
    qd_list = [int(x) for x in args.qdsset.split(",") if x.strip()]
except ValueError:
    raise SystemExit("Invalid --qdsset. Use like: 1,2,4,8,16")

mem_config = get_system_memory_gb()
print(f"--- Detected System Memory: {mem_config} ---")
Path(RESULTS_DIR).mkdir(exist_ok=True)

need_fence = True
try:
    if need_fence:
        set_write_fence()
    print(
        f"\n=== Starting Benchmark for Kernel: {args.name}, "
        f"Type: {args.type} ==="
    )

    if args.type in ["read", "r"]:
        mount_point = get_mount_point(args.file)
        remount(mount_point, "ro") # ro,noatime
        print("Running COLD READ tests (cache cleared before each run)...")
    else:
        pass

```

```
finally:
    if need_fence:
        restore_write_fence()
```

通过上述脚本，单次执行即可完成多个块大小、多种队列深度和不同内核版本之间的顺序读写对比测试，并保持测试前后内核参数一致。

5.1.2 单轮 FIO 执行与缓存控制

在 `run_benchmark.py` 之下，`fio_test.sh` 负责真正调用 `fio` 命令完成单轮测试。`clear_cache.sh` 则在需要 COLD READ 时清空页缓存和相关缓存，使每一轮读测试都从冷缓存起步。这两个脚本尽量保持简单、可读。便于在不同平台（QEMU 虚拟机与树莓派）之间复用。

```
// File: tools/fio_test.sh
#!/usr/bin/env bash
# sudo bash fio_test.sh <title> <bs> <mode> file=<path> ioengine=<engine> \

set -euo pipefail

TITLE="$1"
BS="$2"
MODE="$3"
shift 3

FIO_ARGS=(
    "--name=${TITLE}"
    "--filename=$(echo "$@" | sed -n 's/.*file=\([^ ]*\).*\1/p')"
    "--rw=$(( [ "$MODE" = "r" ] && echo "read" || echo "write" )"
    "--bs=${BS}"
    "--direct=1"
    "--ioengine=$(echo "$@" | sed -n 's/.*ioengine=\([^ ]*\).*\1/p')"
    "--iodepth=$(echo "$@" | sed -n 's/.*iodepth=\([^ ]*\).*\1/p')"
    "--numjobs=$(echo "$@" | sed -n 's/.*numjobs=\([^ ]*\).*\1/p')"
    "--group_reporting=1"
    "--time_based=0"
    "--output=logs/${TITLE}_${BS}_${MODE}_qd$(echo "$@" | sed -n 's/.*qd=\([^ ]*\).*\1/p').log"
)

for arg in "$@"; do
```

```

case "$arg" in
  file=*|ioengine=*|iodepth=*|numjobs=*|qd=*)
    ;;
  *)
    FIO_ARGS+=("$arg")
    ;;
esac
done

mkdir -p logs
echo ">> Running fio with args:"
printf ' %s\n' "${FIO_ARGS[@]}"

fio "${FIO_ARGS[@]}"

// File: tools/clear_cache.sh
#!/usr/bin/env bash

set -euo pipefail

echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
sync
echo "[cache] Dropped pagecache, dentries and inodes."

```

这种拆分主要是为了复现实验。Python 脚本只负责参数展开和日志目录管理，Bash 脚本直接调用 fio；COLD READ 场景下，clear_cache.sh 在每轮测试前清理页缓存。

5.2 性能测试结果与分析

5.2.1 QEMU 虚拟机性能测试

在 QEMU 虚拟机环境中，实验对比了原生 F2FS 内核与引入 Large Folio + iomap 适配后的内核在顺序读写场景下的带宽表现。与第四章的实现动机一致，本节重点关注顺序写、COLD READ 与 WARM READ 三类负载的吞吐变化。

QEMU 实验使用三类测试文件：普通文件、稀疏文件和压缩文件。三类文件使用相同的块大小、队列深度和读写模式。测试文件规模设置为大于内存容量，读路径分别记录 COLD READ 和 WARM READ 的带宽结果。

Bandwidth comparison - normal file - QEMU VM

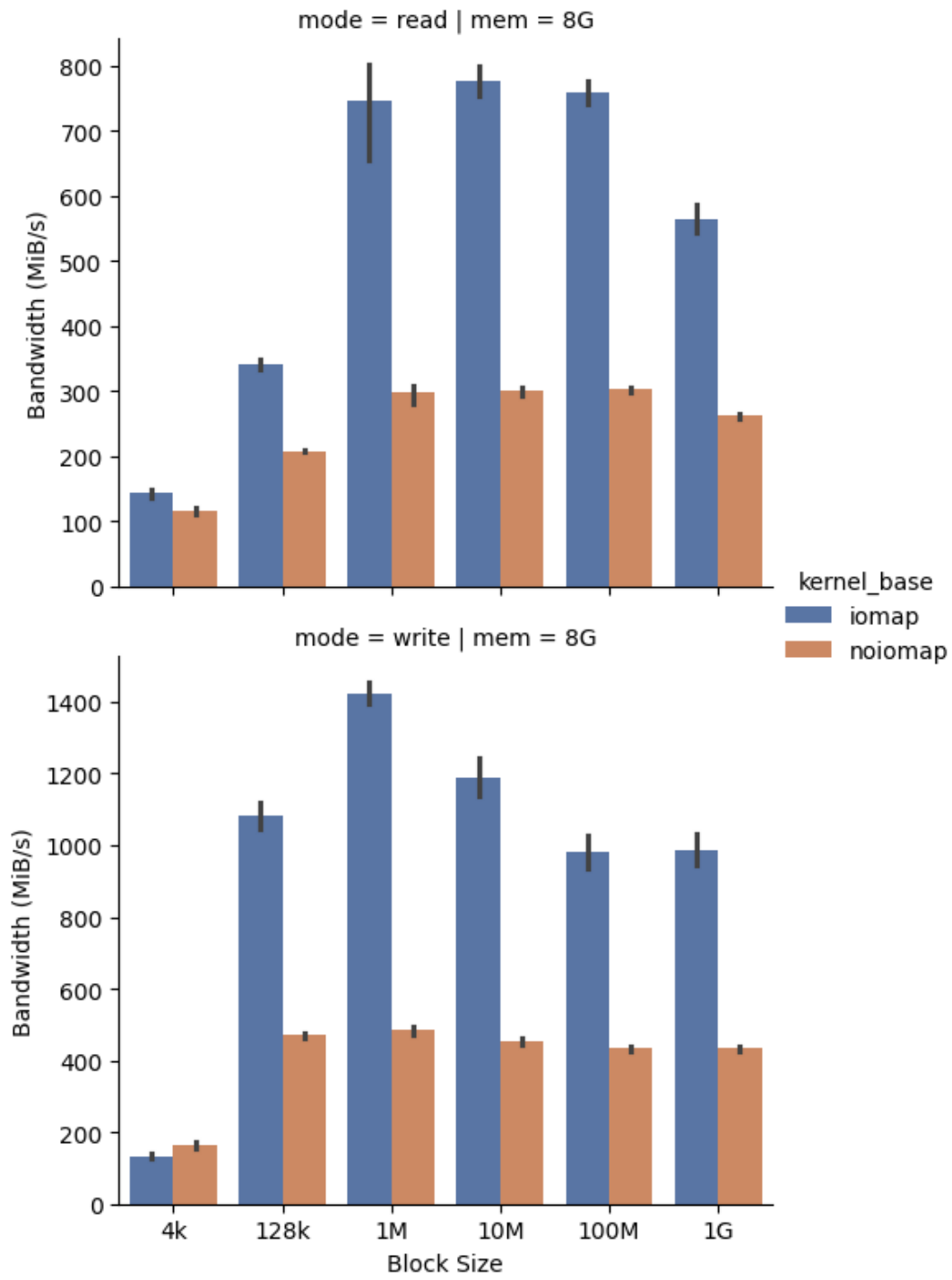


图 5-1 QEMU 虚拟机环境下顺序 I/O 带宽对比（普通文件）

Bandwidth comparison – sparse file (hole) – QEMU VM

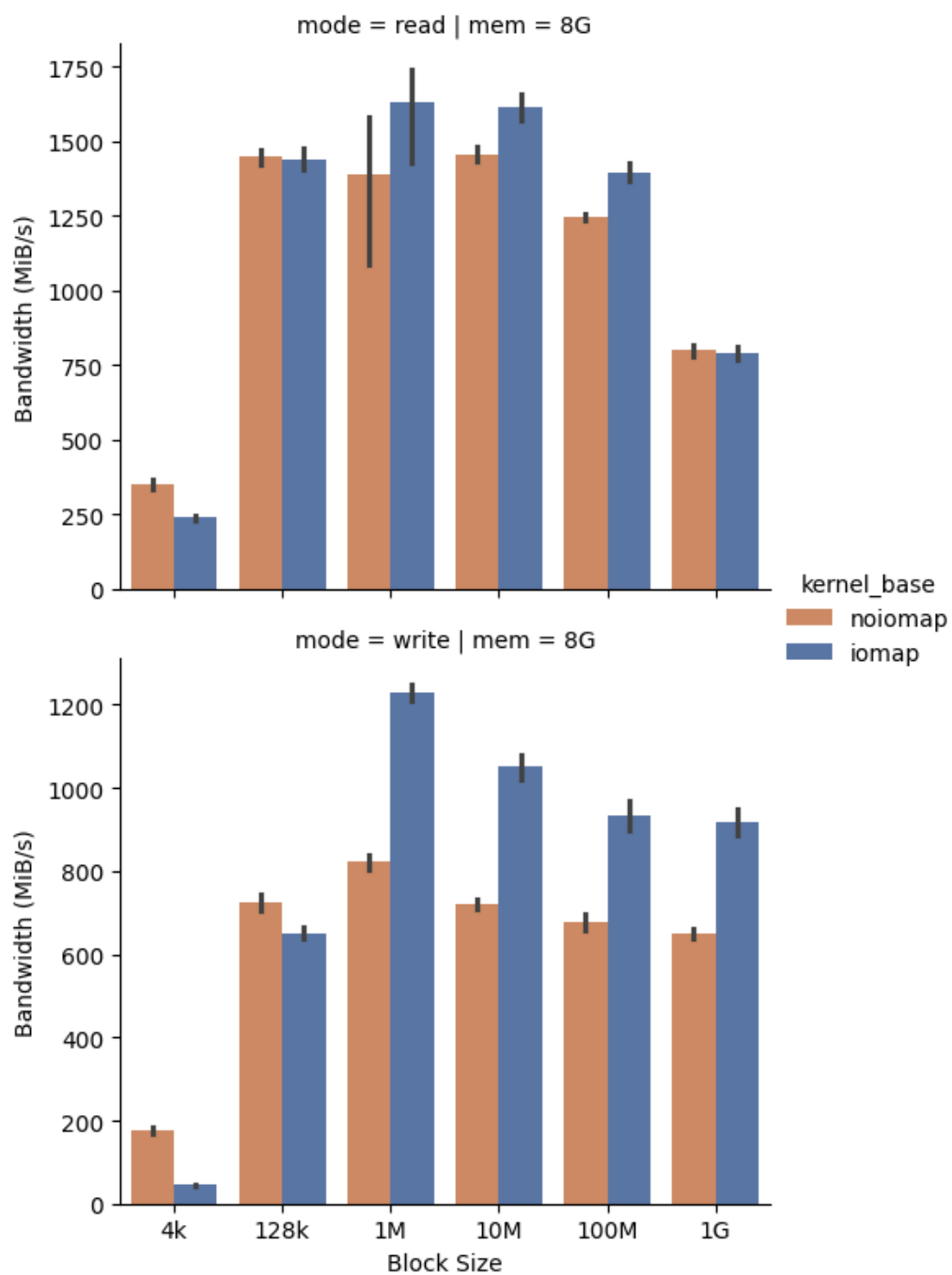


图 5-2 QEMU 虚拟机环境下顺序 I/O 带宽对比（稀疏文件）

Bandwidth comparison - compressed file - QEMU VM

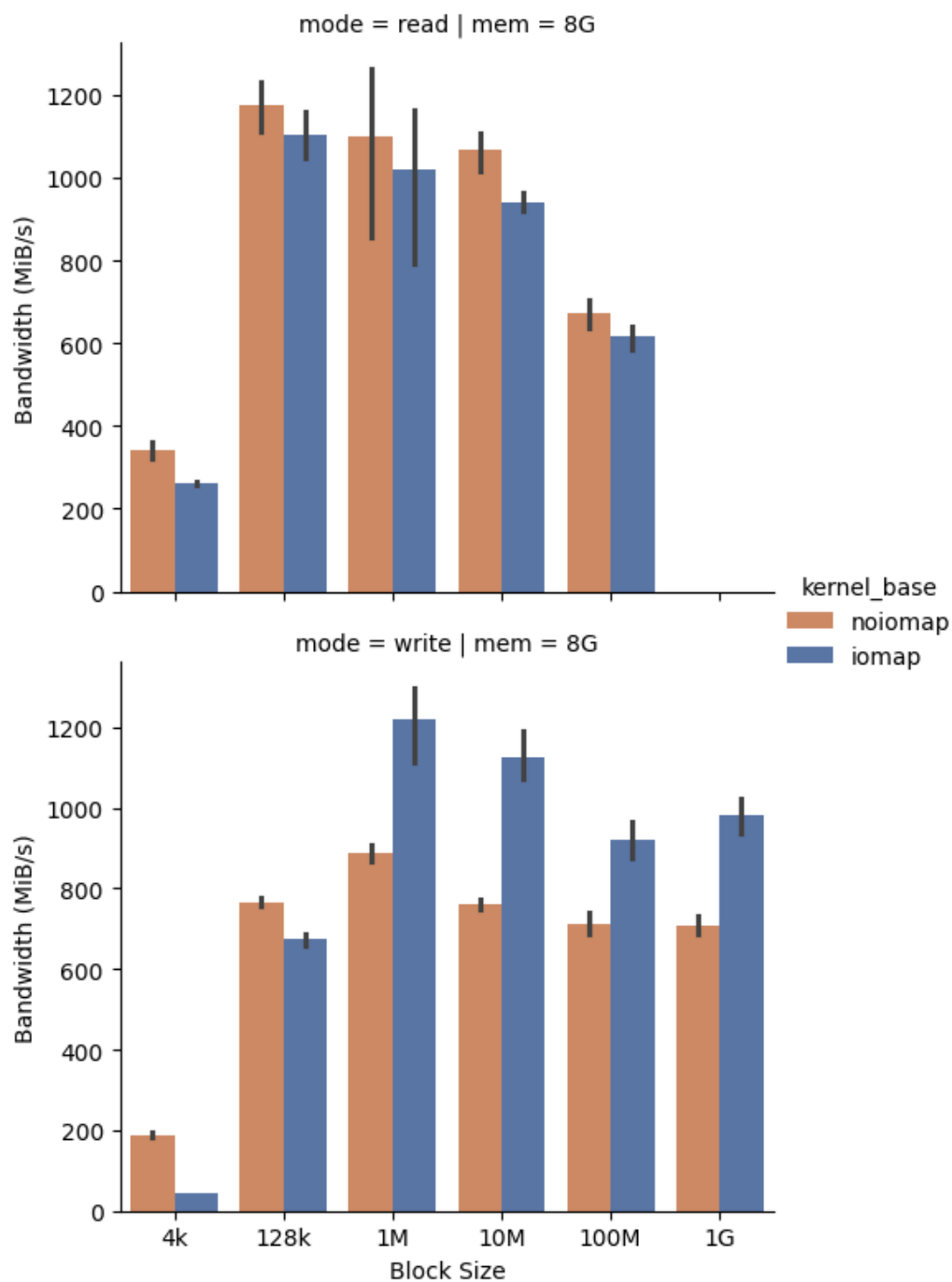


图 5-3 QEMU 虚拟机环境下顺序 I/O 带宽对比（压缩文件）

从结果看，顺序写入场景中改造后路径的带宽高于原始路径。COLD READ 场景下也能观察到读带宽提升，说明新的 `readahead` 和 `folio` 完成逻辑没有成为主要瓶颈。具体数值以图 5-1 至图 5-3 的统计结果为准。

5.2.2 树莓派 5 性能测试

树莓派 5 实验继续使用同一套测试脚本。与 QEMU 不同，该平台通过外接 SSD 运行 F2FS 测试分区，I/O 路径经过真实存储设备和 ARM 处理器。该环境用于检查改动在非虚拟化平台上的表现。

树莓派平台同样测试普通文件、稀疏文件和压缩文件，并分别记录顺序写、COLD READ 和 WARM READ 结果。这样可以直接对照 QEMU 中的同类负载，而无需额外改变测试对象。

Bandwidth comparison - normal file - Raspberry Pi 5

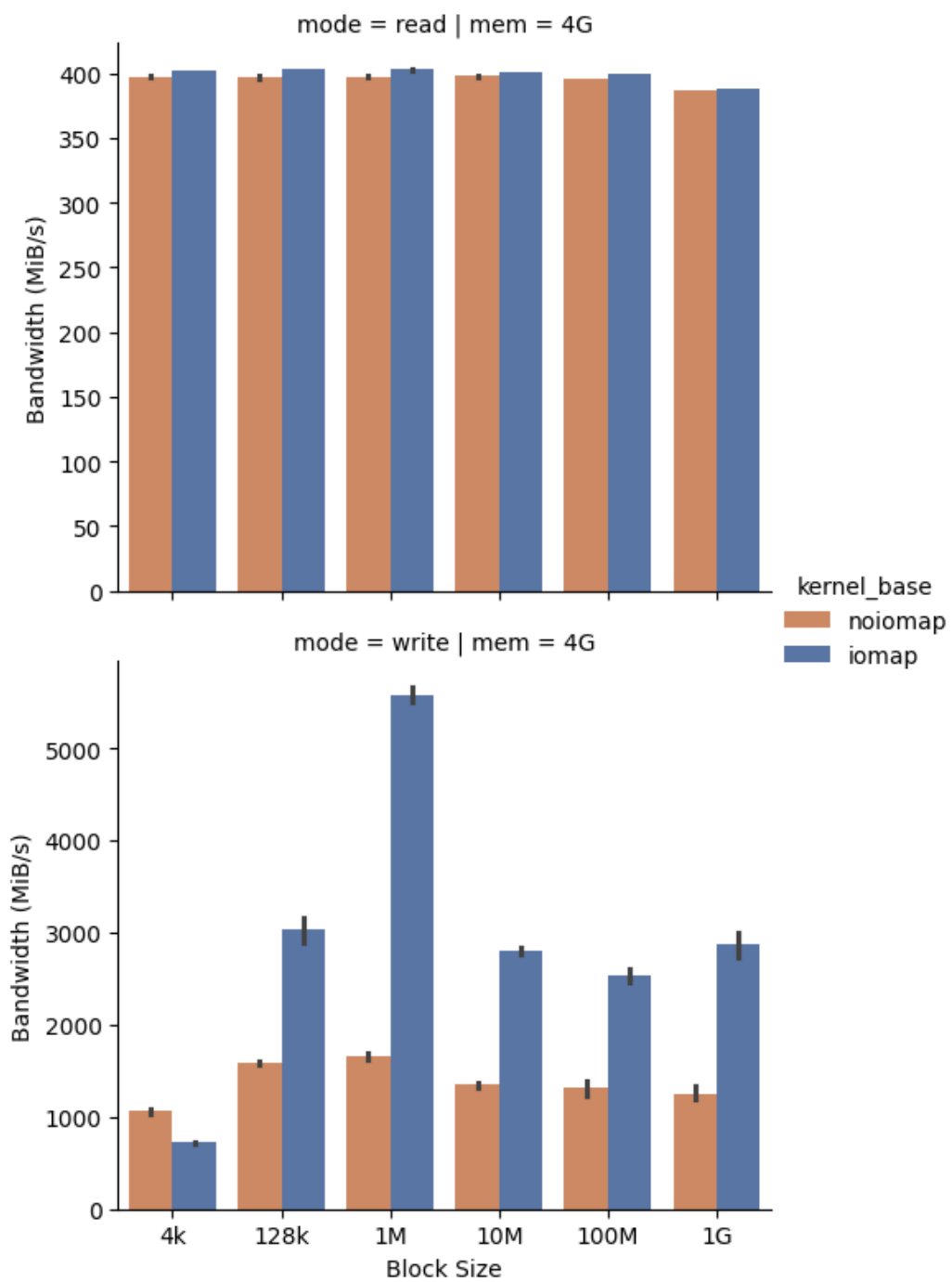


图 5-4 树莓派 5 + SSD (F2FS) 平台顺序 I/O 带宽对比 (普通文件)

Bandwidth comparison - sparse file (hole) - Raspberry Pi 5

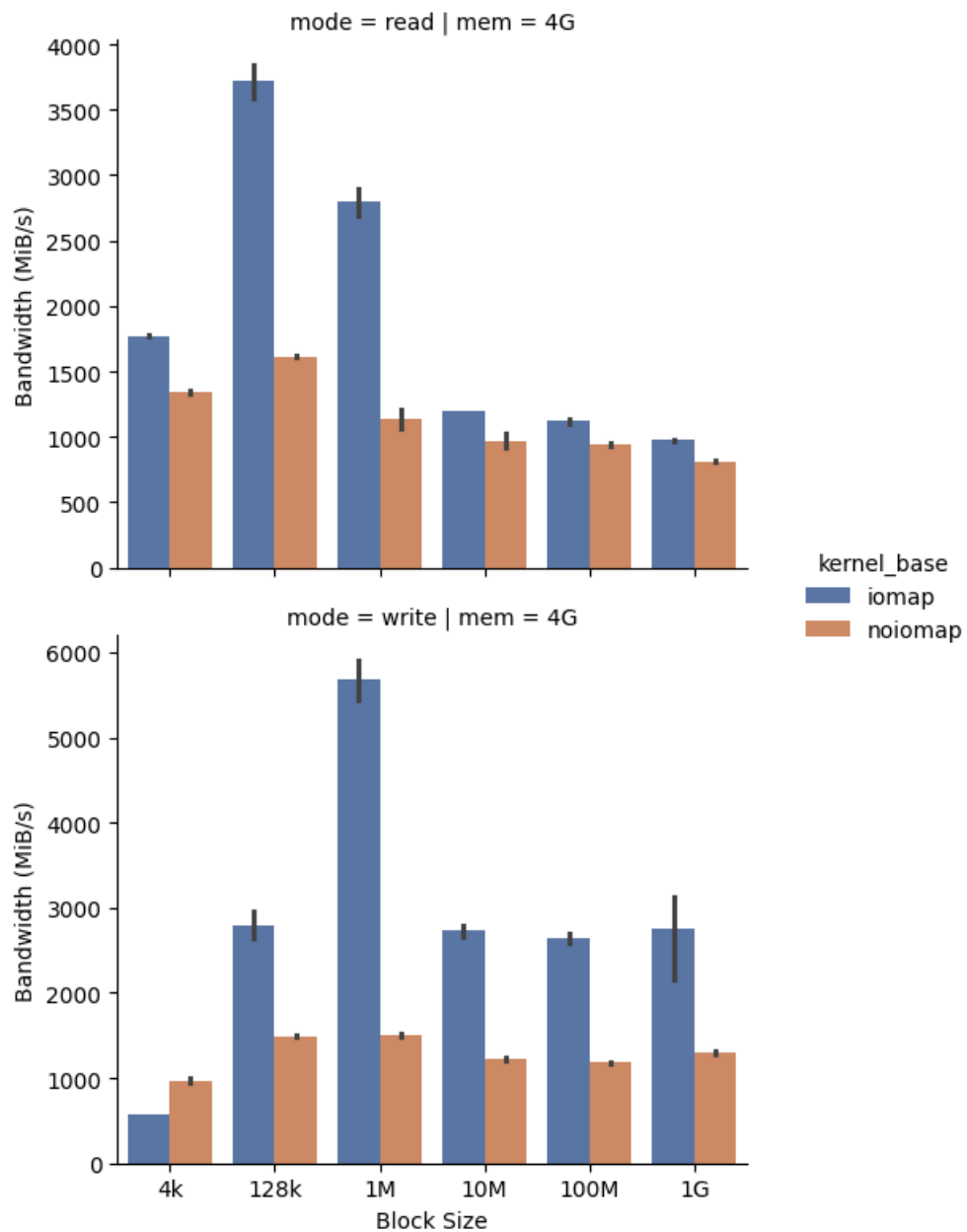


图 5-5 树莓派 5 + SSD (F2FS) 平台顺序 I/O 带宽对比 (稀疏文件)

Bandwidth comparison – compressed file – Raspberry Pi 5

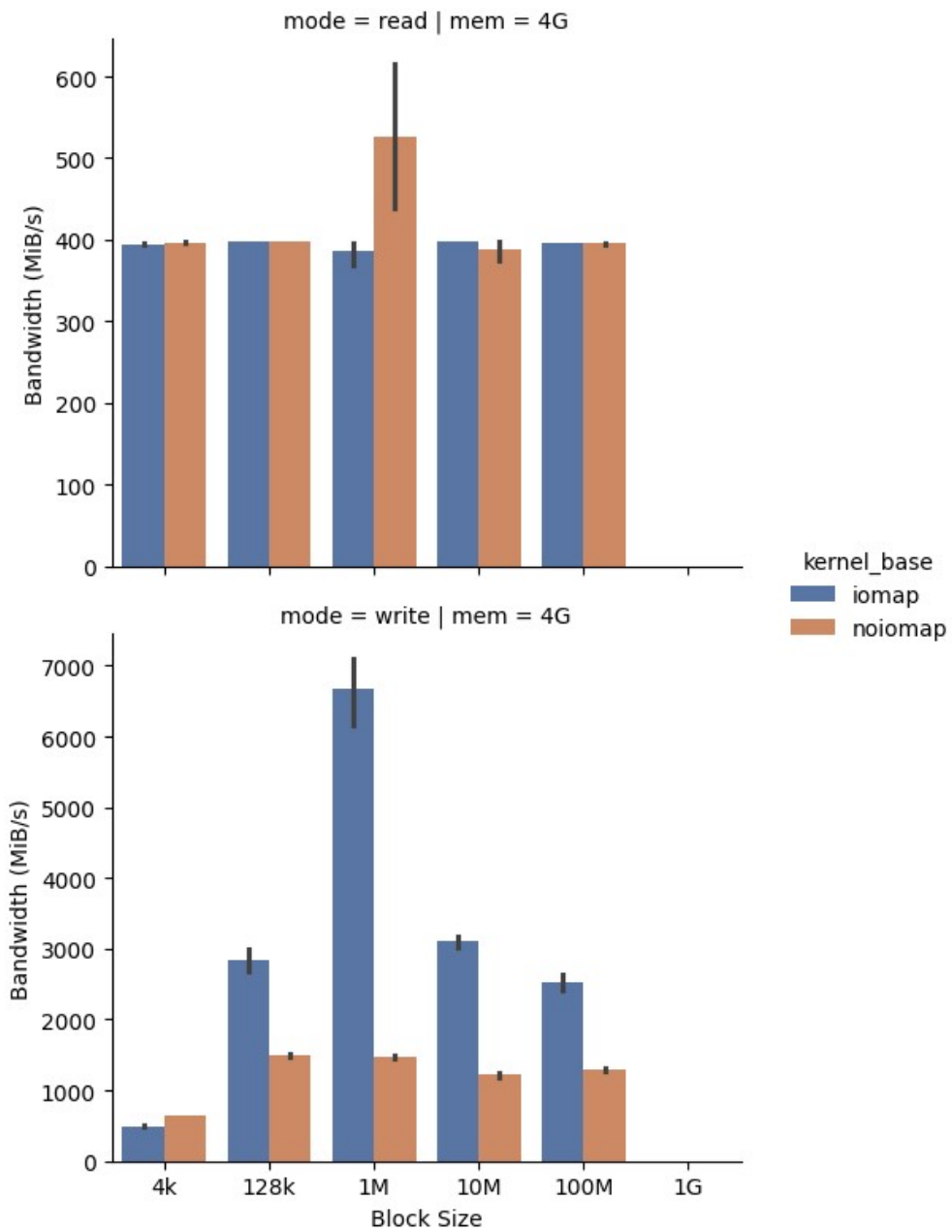


图 5-6 树莓派 5 + SSD (F2FS) 平台顺序 I/O 带宽对比 (压缩文件)

树莓派 5 的结果与 QEMU 结果方向一致。顺序写场景中，改造后路径的带宽高于原始路径；COLD READ 场景下，读路径也受益于更大的页缓存处理粒度。该结果说明，本论文的改动并不依赖虚拟机环境中的缓存行为，在真实 SSD 和 ARM 平台上仍能观察到同方向变化。

5.3 数据处理与可视化脚本

5.3.1 FIO 日志的结构化解析

fio 每轮测试都会生成独立 log 文件。parse_fio_logs.py 从这些文本日志中提取带宽、IOPS、延迟和 CPU 利用率，并写入 CSV，供后续绘图脚本读取。

```
// File: tools/parse_fio_logs.py
#!/usr/bin/env python3

import re
import csv
from pathlib import Path

LOG_DIR = Path("logs")
OUT_CSV = Path("results.csv")

def _parse_with_suffix(s: str) -> float:

    s = s.strip()
    m = re.match(r"([0-9.]+)\s*([kKmMgG]?)", s)
    if not m:
        return float(s)
    val = float(m.group(1))
    suf = m.group(2).lower()
    if suf == "k":
        val *= 1e3
    elif suf == "m":
        val *= 1e6
    elif suf == "g":
        val *= 1e9
    return val

def parse_fio_log(path: Path, meta: dict):
    txt = path.read_text(encoding="utf-8", errors="ignore")

    bw_rg = re.search(r"bw=(\d+(?:\.\d+)?)\s*(MiB/s|GiB/s)", txt, flags=re.I)
```



```

iops_rg = re.search(r"iops=([0-9.kKmMgG]+)", txt, flags=re.I)
cpu_rg = re.search(r"cpu\s*=\s*usr\s*=\s*([0-9.]+)%\s*,\s*sys\s*=\s*([0-9.]+)%",
                    txt, flags=re.I)

clat_rg = re.search(
    r"clat\s*\((usec|msec)\):.*?avg=\s*([0-9.]+)",
    txt, flags=re.I | re.S
)
p99_rg = re.search(
    r"99.00th=\s*([0-9.]+)",
    txt, flags=re.I
)

if not (bw_rg and iops_rg and cpu_rg):
    print(f"[warn] 必要字段缺失 → {path.name}")
    return None

bw = float(bw_rg.group(1))
if "GiB/s" in bw_rg.group(2):
    bw *= 1024.0

iops = _parse_with_suffix(iops_rg.group(1))

clat_avg = None
clat_p99 = None
if clat_rg:
    clat_avg = float(clat_rg.group(2))
    if clat_rg.group(1).lower() == "msec":
        clat_avg *= 1000.0

if p99_rg:
    clat_p99 = float(p99_rg.group(1))
    if "clat percentiles (msec)" in txt:
        clat_p99 *= 1000.0

cpu_usr = float(cpu_rg.group(1))
cpu_sys = float(cpu_rg.group(2))

return {
    **meta,
    "bw_mib_s": bw,

```

```

    "iops": iops,
    "clat_avg_us": clat_avg,
    "clat_p99_us": clat_p99,
    "cpu_usr_pct": cpu_usr,
    "cpu_sys_pct": cpu_sys,
}

def main():
    rows = []
    for p in LOG_DIR.glob("*.log"):
        name = p.stem
        meta = {"raw_name": name}
        r = parse_fio_log(p, meta)
        if r:
            rows.append(r)

    if not rows:
        print("[warn] 未解析到任何有效行")
        return

    keys = sorted(rows[0].keys())
    with OUT_CSV.open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=keys)
        w.writeheader()
        w.writerows(rows)
    print(f"[ok] 写出 {len(rows)} 行到 {OUT_CSV}")

if __name__ == "__main__":
    main()

```

解析脚本只保留绘图需要的字段，避免后续图表直接依赖 fio 原始文本格式。

5.3.2 测试结果的图表生成

visualize_results.py 从 CSV 中读取结构化结果，并按内核版本、块大小和队列深度分组。脚本使用 pandas 完成聚合，再用 matplotlib 输出带宽和 CPU 使用率等图表。

```
// File: tools/visualize_results.py
```

```
#!/usr/bin/env python3

import pandas as pd
import matplotlib.pyplot as plt

CSV_PATH = "results.csv"
OUT_DIR = "figs"

def plot_bw_by_kernel(df: pd.DataFrame, title_suffix: str, out_name: str):

    grouped = df.groupby(["kernel", "bs"])["bw_mib_s"].mean().reset_index()
    piv = grouped.pivot(index="bs", columns="kernel", values="bw_mib_s")

    ax = piv.plot(kind="bar")
    ax.set_xlabel("Block size")
    ax.set_ylabel("Bandwidth (MiB/s)")
    ax.set_title(f"Sequential throughput ({title_suffix})")
    ax.legend(title="Kernel")

    plt.tight_layout()
    plt.savefig(f"{OUT_DIR}/{out_name}.png")
    plt.close()

def main():
    df = pd.read_csv(CSV_PATH)
    Path(OUT_DIR).mkdir(exist_ok=True)

    df_read = df[df["mode"] == "read"]
    plot_bw_by_kernel(df_read, "read", "bw_read")

    df_write = df[df["mode"] == "write"]
    plot_bw_by_kernel(df_write, "write", "bw_write")
```

```
if __name__ == "__main__":  
    main()
```

第五章的数据链路由三部分组成：fio 负责产生原始日志，parse_fio_logs.py 负责结构化解析，visualize_results.py 负责生成图表。本章后续结论均以这一链路得到的数据为依据。

结语

本文围绕 F2FS 支持 Large Folios 的工程适配展开。首先，论文分析了 4 KiB page 粒度与 F2FS block、压缩簇和 GC 搬移范围之间的粒度差异；随后，在实现层面引入 iomap 区间映射，扩展 folio 私有状态，并调整读完成、写回子范围提交、GC 搬运和压缩文件路径。QEMU 与树莓派 5 平台上的顺序读写测试表明，改造后路径在大块 I/O 场景下能够获得更高吞吐。该结果说明，Large Folios 与 iomap 可以作为 F2FS 页缓存路径继续优化的实现基础。

参考文献

- [1] Weisberg P, Wiseman Y. Using 4KB Page Size for Virtual Memory is Obsolete[A]. IEEE International Conference on Information Reuse and Integration, Proceedings[C]. Las Vegas: IEEE, 2009: 1–8.
- [2] Jia W W, Zhang J Y, Shan J C, Ding X N. Effective Huge Page Strategies for TLB Miss Reduction in Nested Virtualization[J]. IEEE Transactions on Computers, 2024, 73(8): 1983–1996. DOI: 10.1109/TC.2024.3398498.
- [3] Panwar A, Gupta A, Srikantaiah S. Making Huge Pages Actually Useful[A]. ASPLOS 2018 Conference Proceedings[C]. New York: ACM, 2018: 679–692.
- [4] Liu L, Li X, Yang S, et al. Thinking about a New Mechanism for Huge Page Management[A]. Proceedings of the 10th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '19)[C]. New York: ACM, 2019: 1–7.
- [5] Patil A. TLB and Pagewalk Performance in Multicore Architectures with Large Die-Stacked DRAM Cache[J]. arXiv preprint arXiv:2002.01073, 2020.
- [6] Sites R L. Larger Pages[EB/OL]. <https://www.sigarch.org/larger-pages>, 2022-05-10.
- [7] Martins J. Minimizing struct page overhead[EB/OL]. <https://blogs.oracle.com/linux/minimizing-struct-page-overhead>, 2021-04-08.
- [8] Corbet J. Pulling slabs out of struct page[EB/OL]. <https://lwn.net/Articles/871982>, 2021-10-08.
- [9] Corbet J. The multi-generational LRU[EB/OL]. <https://lwn.net/Articles/851184>, 2021-04-02.
- [10] Zhao Y. Multigenerational LRU Framework[EB/OL]. <https://lore.kernel.org/all/20220204053230.2832968-1-yuzhao@google.com>, 2022-02-04.
- [11] Corbet J. The state of the page in 2023[EB/OL]. <https://lwn.net/Articles/931794/>, 2023-05-17.
- [12] Corbet J. Toward the unification of hugetlbfs[EB/OL]. <https://lwn.net/Articles/974491>, 2024-05-22.
- [13] Corbet J. Measuring memory fragmentation[EB/OL]. <https://lwn.net/Articles/969246>, 2024-05-28.
- [14] Corbet J. The buffer_head structure[EB/OL]. <https://lwn.net/Articles/322674>, 2009-03-11.
- [15] Wong D J. Supported File Operations[EB/OL]. <https://docs.kernel.org/filesystems/iomap/operations.html>, 2024-03-01.
- [16] Wong D J. Large Block-Size Support[EB/OL]. <https://docs.kernel.org/filesystems/iomap/large-block-size.html>, 2024-03-01.
- [17] Tang C Q. Contiguitas: Top Picks in Memory Contiguity Research[EB/OL]. <https://tangchq74.github.io/Contiguitas-Top-Picks.pdf>, 2023-09-01.
- [18] Lee C, Sim D, Hwang J, et al. F2FS: A New File System for Flash Storage[A]. Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST '15)[C]. Santa Clara: USENIX Association, 2015: 273–286.
- [19] Linux Kernel Documentation. Flash-Friendly File System (F2FS)[EB/OL]. <https://docs.kernel.org/filesystems/f2fs.html>, 2026-05-07.
- [20] Choi J M. F2FS Design[EB/OL]. https://embedded.dankook.ac.kr/~choijm/course/202302AOS/20230926_F2FS.pptx, 2023-09-26.

[21] Wang X, Yang B. Optimizing the Cost of Garbage Collection in F2FS Using Working Set Strategy[J]. Highlights in Science, Engineering and Technology, 2023, 39: 896–901. DOI: 10.54097/hset.v39i.6673.

[22] Google LLC. Support 16 KB page sizes[EB/OL]. <https://developer.android.com/guide/practices/page-sizes>, 2026-05-07.

致谢

感谢我的导师在操作系统方面的悉心教学以及选题方面的指导,让我拥有了扎实的基本功以及明确的努力方向,并且感谢导师的持续鼓励,让我在面对晦涩复杂的 Linux 内核代码能鼓起勇气和信心,坚持到底。